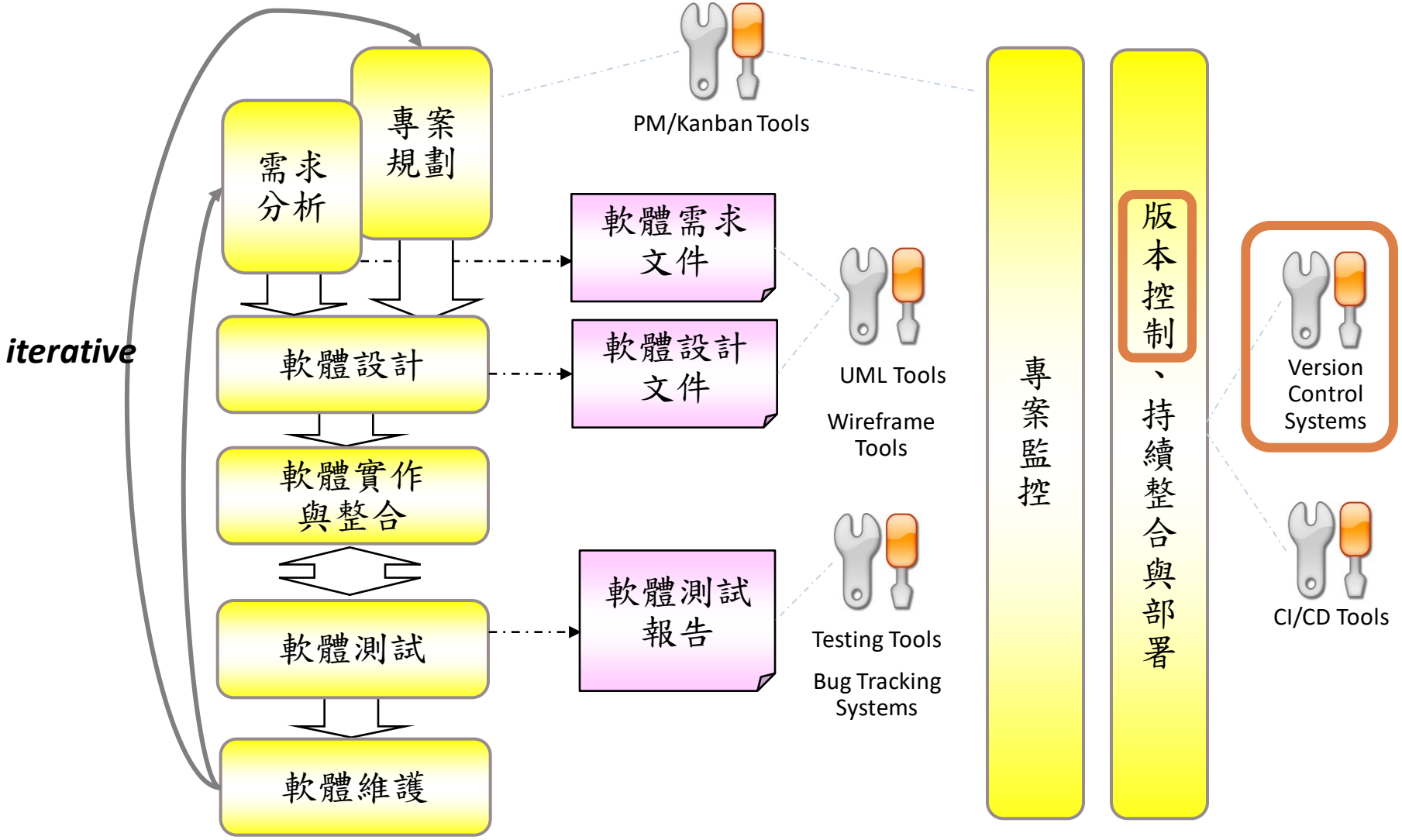
軟體工程課程:
版本控制
(Version Control)

海大資工 馬尚彬

單元大綱

- 版本控制概念
- GIT基本指令
- GIT分支作法
- 使用遠端儲存庫GitHub
- GitHub Pull Request
- Git/GitHub開發流程

軟體開發步驟、文件與工具

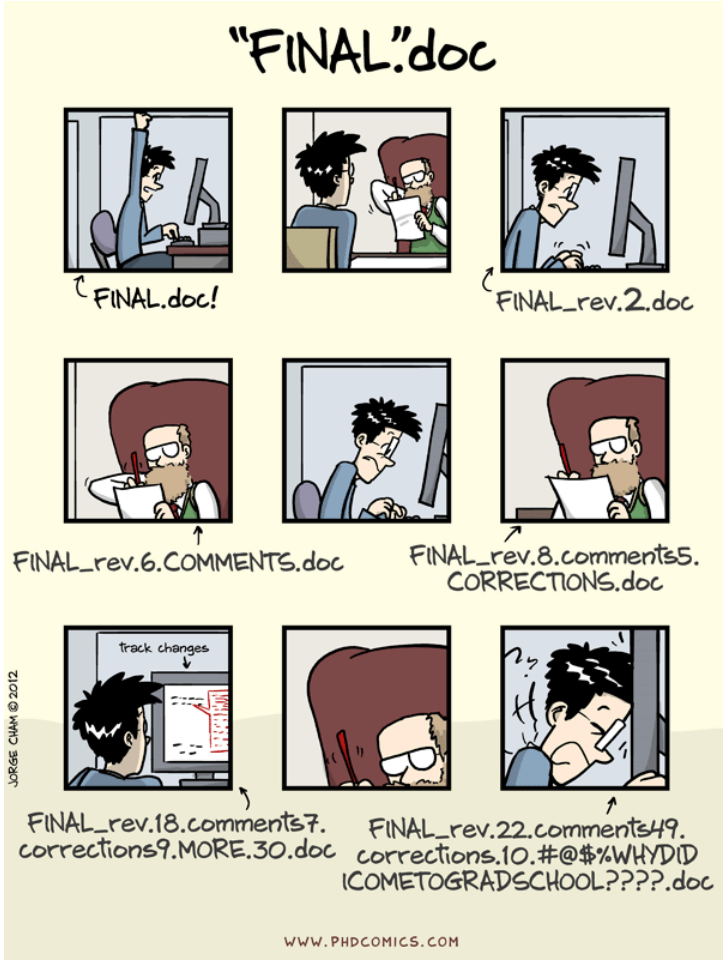


建構管理

- 建構管理(Software Configuration Management, SCM)的主要目標是確保軟體開發和維護過程中的一致性、可追溯性和可重複性，包含：
  - 版本控制：管理原始程式碼和相關文件的版本，跟蹤變更歷史，以便能夠準確地回溯到先前的狀態。(Git/GitHub, SVN)
  - 組態控制：確保軟體系統的各個元件和組件按照規定的標準進行配置和管理。(Chef, Puppet, Ansible)
  - 建構自動化：建立自動化流程來執行建構、測試和部署軟體，以確保產品的品質。(Jenkins, GitHub Actions, Travis CI)
  - 發布管理：管理軟體產品的發布流程，包括版本號管理、發布策略、更新管理等。

為什麼需要版本控制？

- 當你撰寫報告時，如果你想備份，你會怎麼做？



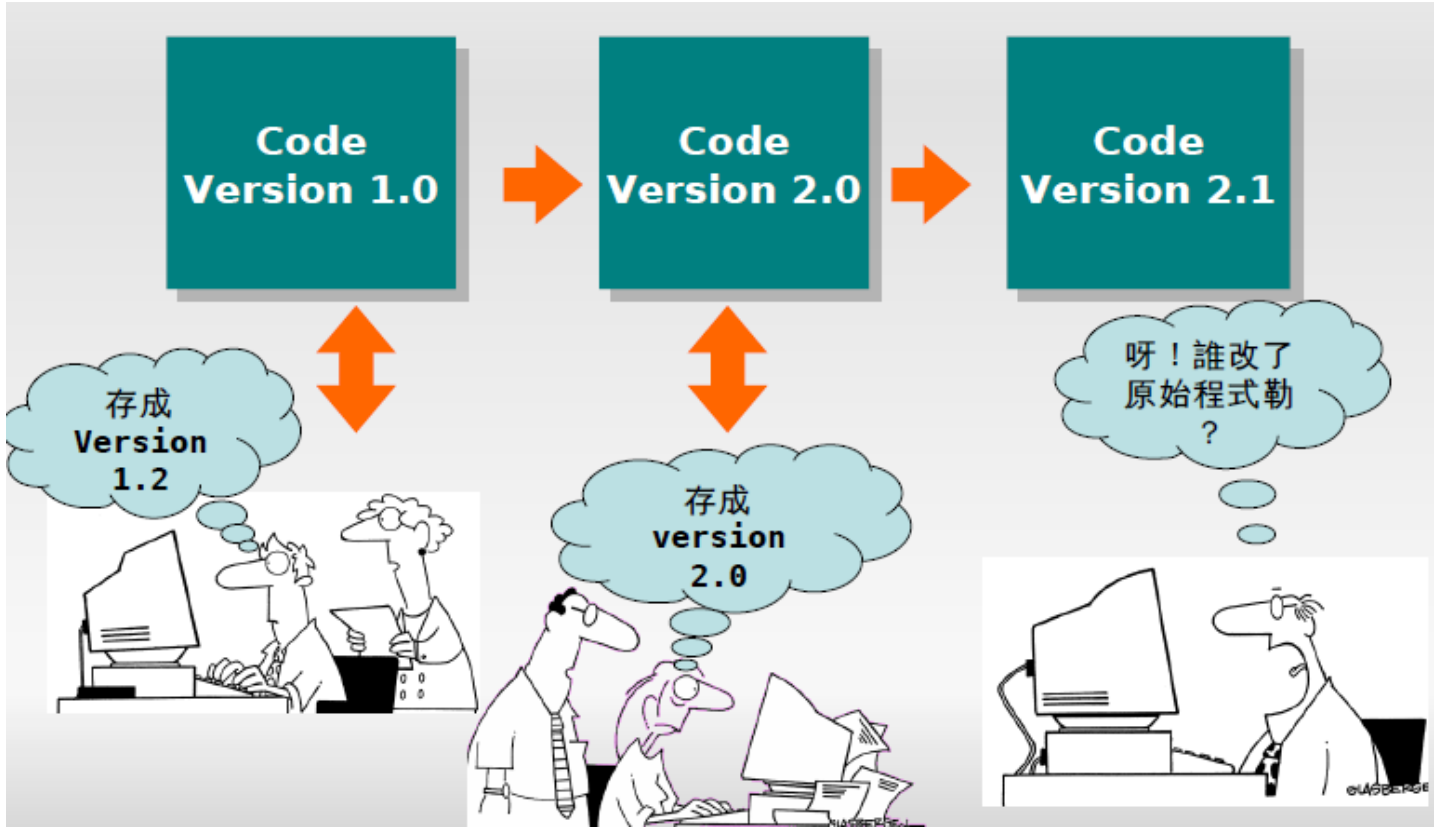
會發生什麼事？

- 很多版本檔案



會發生什麼事？

- 跟別人合作時.....

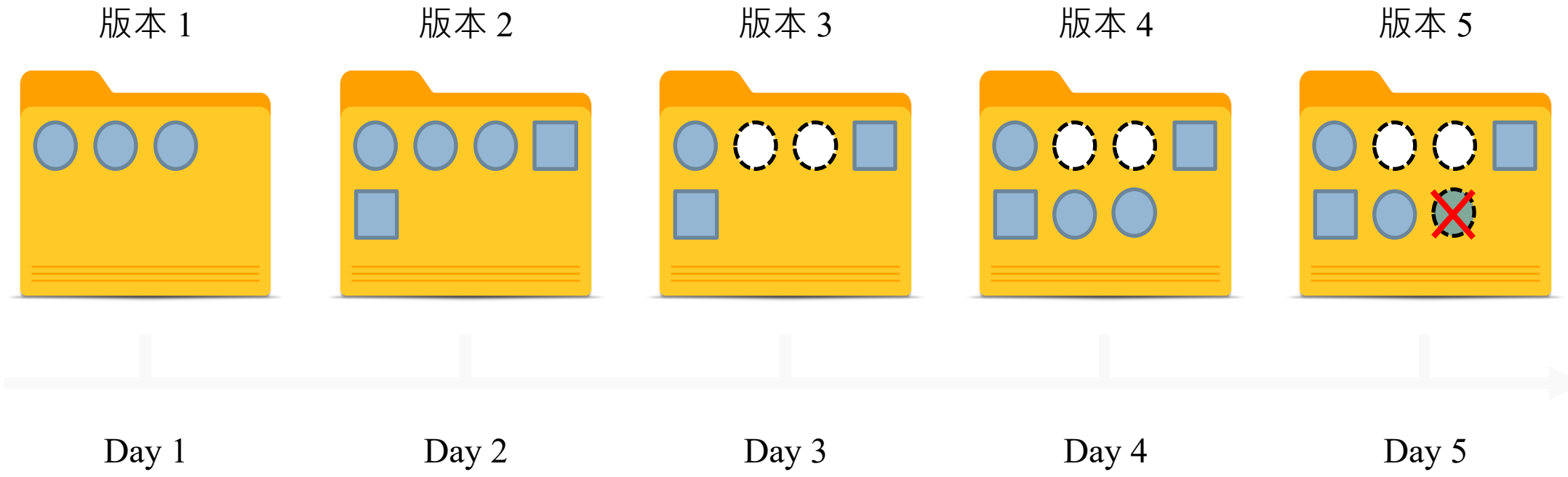


資料來源:自由軟體鑄造場王家薰，如何使用OpenSource的工具-協助軟體的開發及快速架站

為什麼要版本控制

- 「凡走過必留下痕跡」
  - 追蹤歷程
  - 改了東西，不會改不回來
- 「三個臭皮匠勝過一個諸葛亮」
  - 大家一起改，不會互相干擾
  - 大家一起改，還能清楚知道對方改了什麼

什麼是「版本」？



甚麼是GIT?

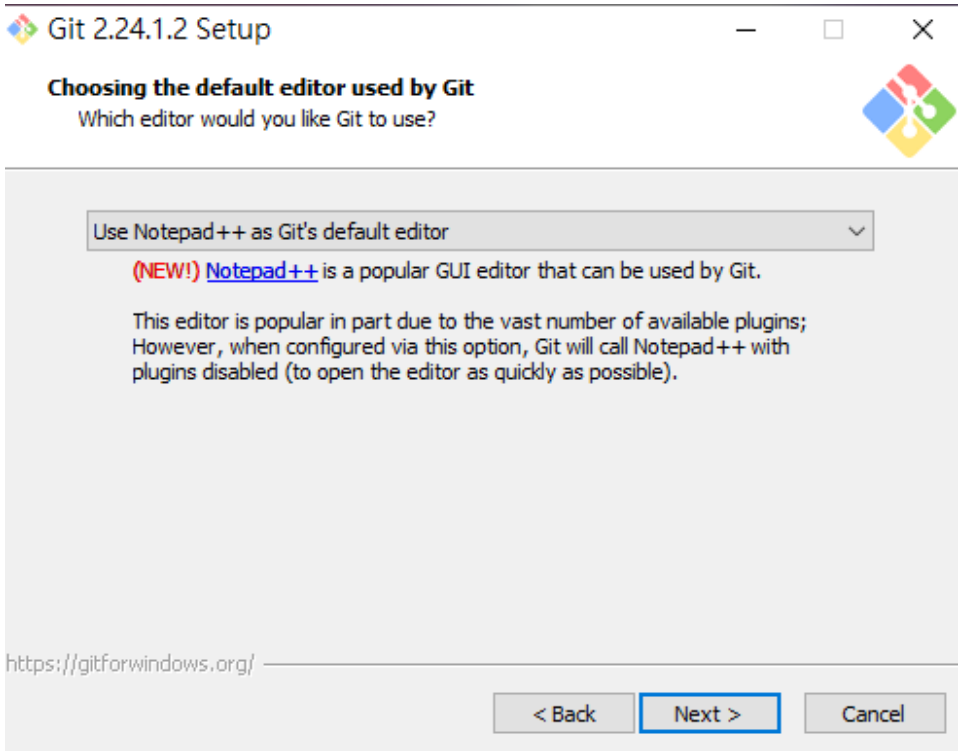
- Git 是一種分散式版本的版本控制(Version Control)系統
- 甚麼是版本？
  - 你的(軟體)專案，不管是新增或刪除檔案，亦或是修改檔案內容，都稱之為一個「版本」
  - 「版本控制系統」，會記錄這些所有的狀態變化，並且可以像搭乘時光機一樣，隨時切換到過去某個「版本」時候的狀態。

Git的優點

- 1. 免費、開源
  - Git 是由 Linux 核心的作者 Linus Torvalds 在 2005 年為了管理 Linux 核心程式碼，僅花了 10 天所開發出來的
- 2. 速度快、檔案體積小
  - Git 特別的設計，在於它並不是記錄版本的差異，而是記錄檔案內容的「快照」（snapshot)。
- 3. 分散式系統
  - Git 是一款分散式的版控系統（Distributed Version Control）。
  - 大多的 Git 操作也都是在自己電腦本機就可以完成。

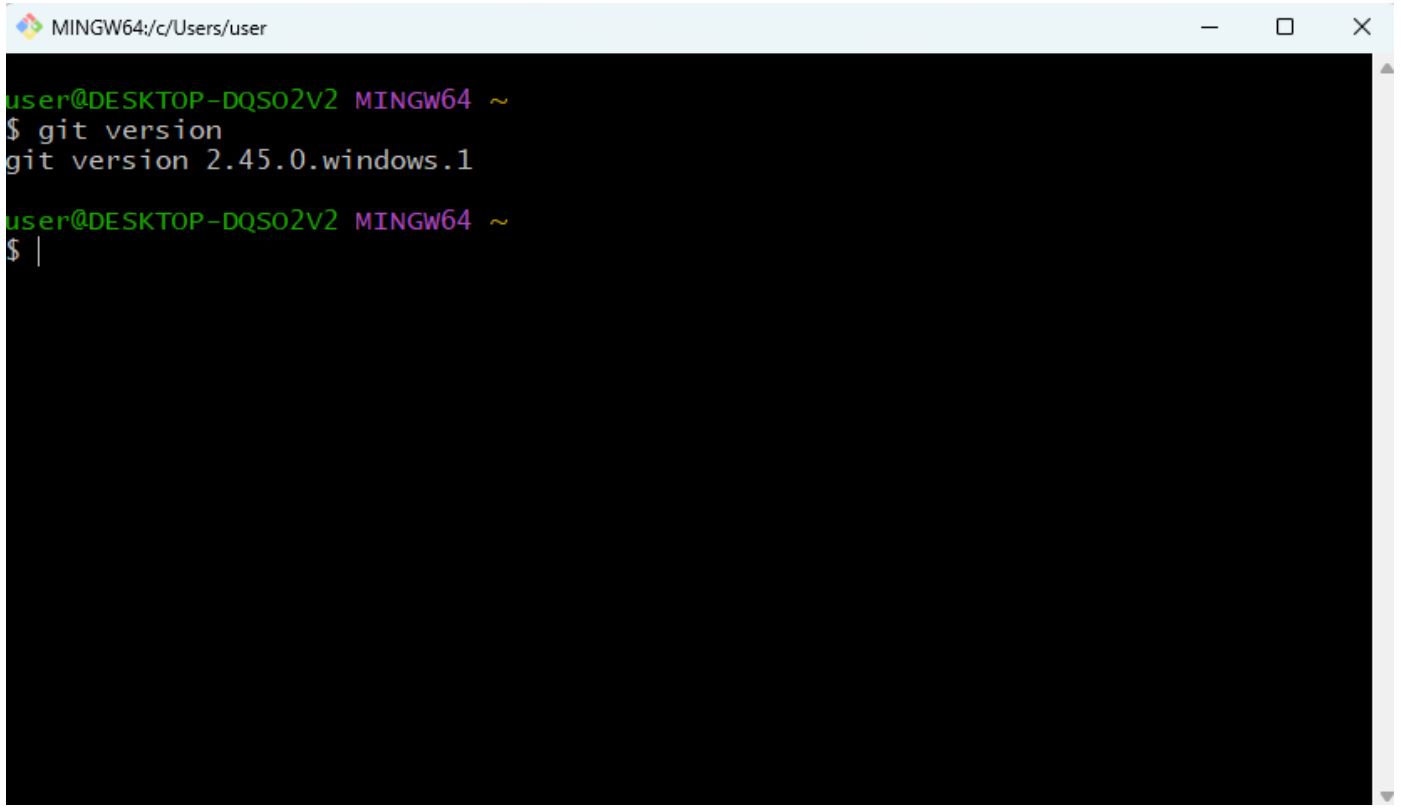
Git系統安裝

- 請到官方網站下載合適的Git版本：
  - https://git-scm.com/download/win
  - 安裝過程中可以更換預設編輯器為Notepad++或其他你熟悉的編輯器



Git Bash

- 安裝完成之後，請啟動「Git Bash」
  - Windows系統版本模擬了一個在Linux 世界的Bash



可以右鍵選[Options]->[Text]
改字型(Font)大小

可以試打指令看看：
git version

常用終端機命令列指令

指令	說明
cd	切換目錄
pwd	取得目前所在的位置
ls	列出目前的檔案列表
mkdir	建立新的目錄
touch	建立檔案
cp	複製檔案
mv	移動檔案
rm	刪除檔案
clear	清除畫面上的內容

設定Git

- 要開始使用 Git，首先要設定使用者的 Email 信箱以及使用者名稱：

```
user@DESKTOP MINGW64 /e/myprj
$ git config --global user.name "Albert Ma"

user@DESKTOP MINGW64 /e/myprj
$ git config --global user.email "albert@ntou.edu.tw"
```

- 可以看看Git的目前設定

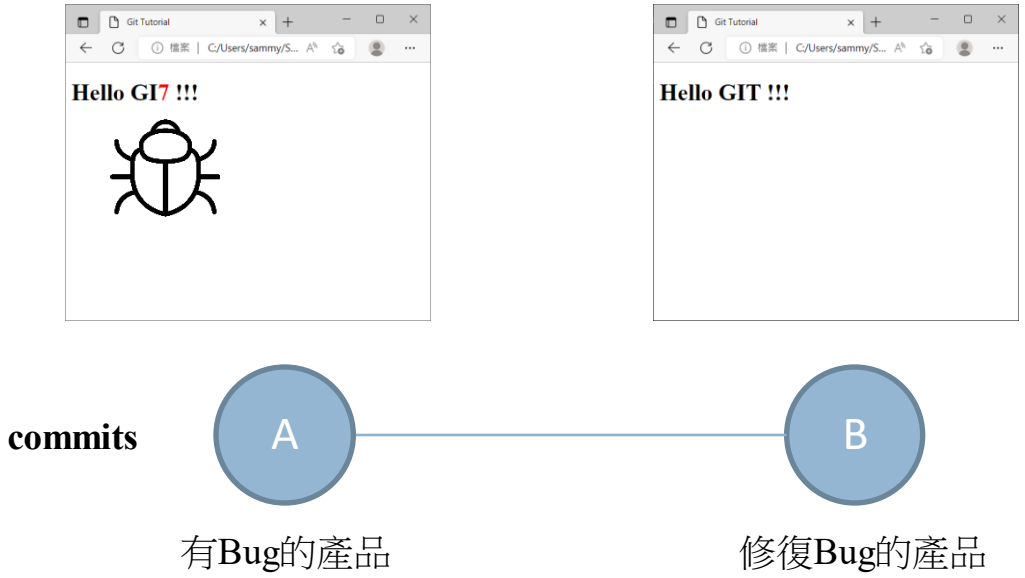
```
user@DESKTOP MINGW64 /e/myprj
$ git config --list
```

(移除設定： $ git config --global --unset XX.YY)

(user@DESKTOP是老師的帳號與機器名稱)

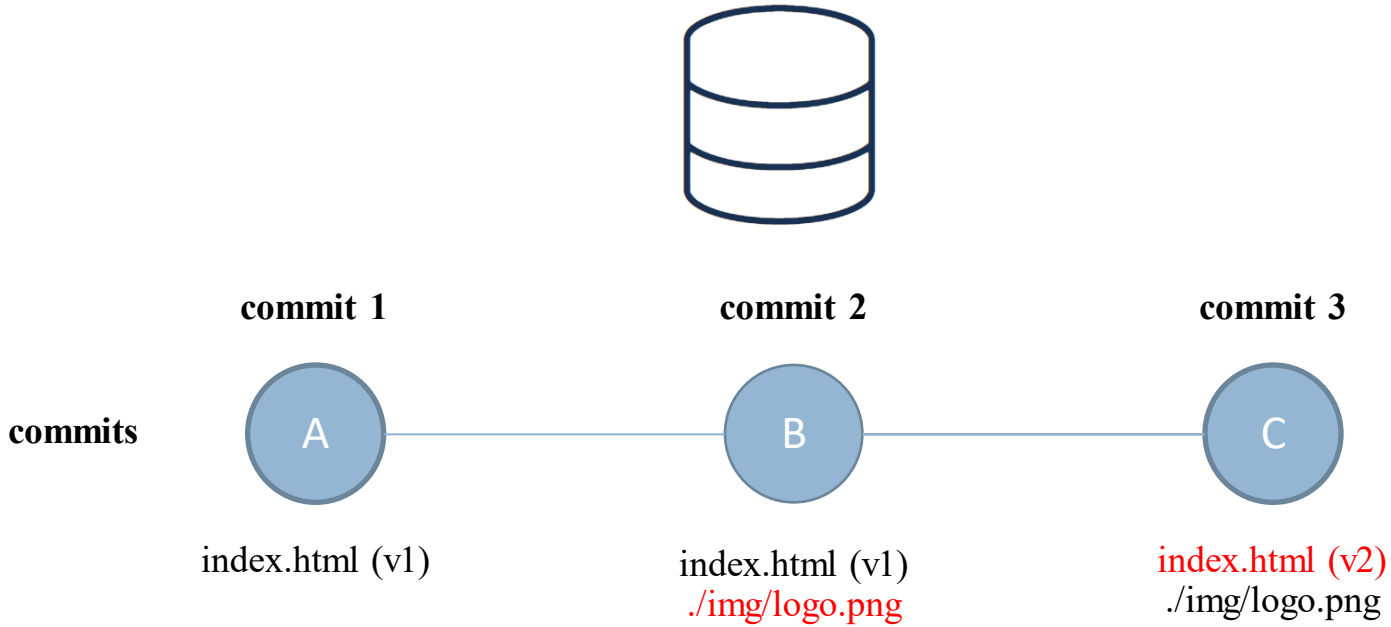
Git Commits

- Git記錄專案在不同時間的狀態，構成一個專案開發的歷程
- 專案中每個記錄版本稱為一個提交(commit)
- 每一次commit可視為專案在一個給定的時間點上的快照(snapshot)



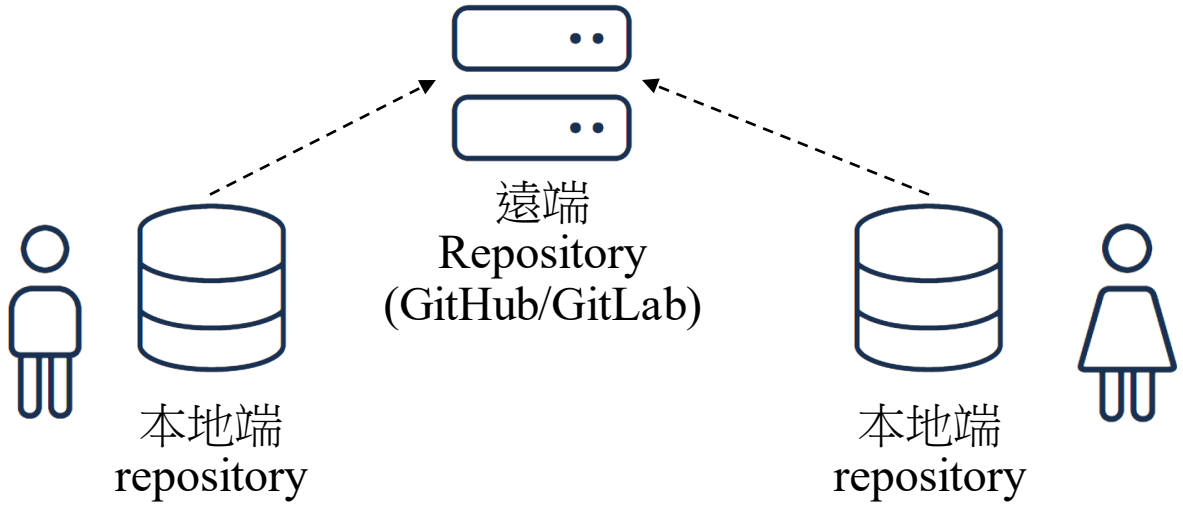
Git Repository (Repo) 是什麼?

- 一連串的專案快照(snapshot)，包含多個提交(commit)



分散式版本控制系統

- 分散式版本控制系統具有三特性
  - 每一位使用者皆有自己本地端的專案歷程
  - 使用者可以離線使用版控系統
  - 可以方便地進行儲存庫(Repository)內容的整合工作



新增與初始Repository

- 建立目錄
  - Git預設工作目錄是在C:\Users\{你的名字}
  - 可以先以檔案總管建立專案目錄(用命令列也可)
  - 再切換至專案目錄(如底下範例是simple專案目錄)
- 使用 git init 指令初始化這個目錄
  - 讓 Git 開始對這個目錄進行版本控制

```
user@DESKTOP MINGW64 ~
$ cd gittest

user@DESKTOP MINGW64 ~/gittest
$ git init
Initialized empty Git repository in
C:/Users/Albert/gittest/.git/
```

查看目錄狀態

- *git status*指令: 查詢現在這個目錄的「狀態」。

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

- 若我們把檔案複製到這個目錄，再執行git status
  - 會多看到「 Untracked files 」資訊(未被「追蹤」)

將檔案交給Git追蹤1

- 讓 Git 開始「追蹤」檔案：
  - 用*git add* 指令後面加上檔案名稱
  - 可再執行git status看狀態

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git add MyFirst.java

user@DESKTOP MINGW64 ~/gittest (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   MyFirst.java
```

剛才那個檔案從Untracked 變成new file 狀態了。表示這個檔案已經被安置到暫存區(Staging Area，也被稱為index)。

將檔案交給Git追蹤2

- 使用萬用字元 (wildcard character)：
  - $ git add *.java
  - $ git add *.* (目前目錄層全部檔案)
- --all 參數 (把全部的檔案加到暫存區)：
  - $ git add --all (連子目錄都會加入)

把暫存區的內容提交到倉庫

- *git commit*指令
  - 讓暫存區的內容永久的存下來(才算正式進入到版本控制)

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git commit -m "init commit"
[master (root-commit) f34a359] init commit
 1 file changed, 57 insertions(+)
 create mode 100644 MyFirst.java
```

  - -m 參數:
    - 在 Commit 的時候，要以-m參數輸入訊息(message)。
    - 主要的目的就是告訴你自己以及其它人「這次的修改做了什麼」。
    - 如果沒有給，Git會開預設的編輯器(如Notepad++)讓你寫訊息。
    - 如果檔案中還是沒加訊息，git不會讓commit成功執行。

約定式提交(Conventional Commit)

- Conventional Commit: 提供一些簡單的條件集合用於建立明確的提交歷史
- Commit Types: feat, fix, chore, docs, refactor, test, etc.
- 範例：

```
feat: allow provided config object to extend other configs

BREAKING CHANGE: `extends` key in config file is now used for extending other config files
```

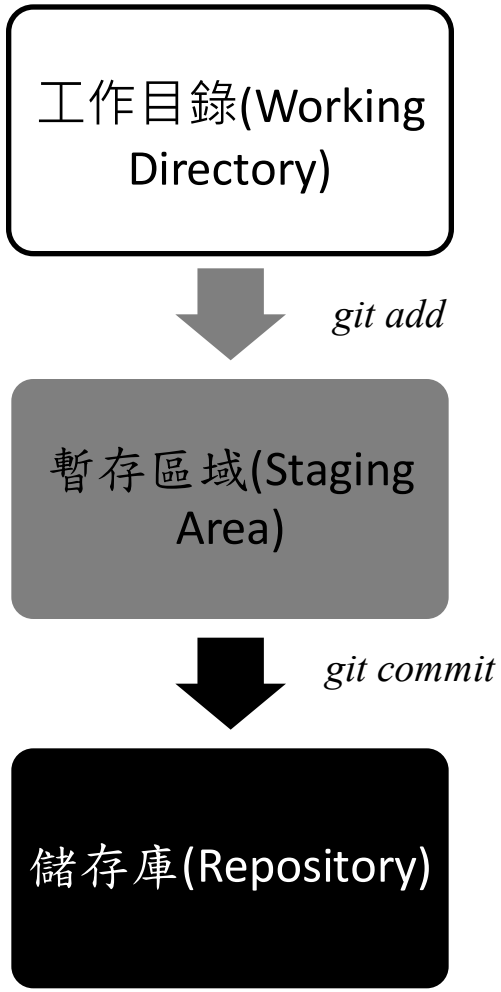
```
fix: correct minor typos in code

see the issue for details on the typos fixed

closes issue #12
```

https://www.conventionalcommits.org/en/v1.0.0/
https://www.conventionalcommits.org/zh-hant/v1.0.0-beta.4/

工作區、暫存區與儲存庫1



- Git有「工作目錄(Working Directory)」、「暫存區(Staging Area)」、「儲存庫(Repository)」三個區塊。
  - 透過不同的 Git 指令，可以把檔案移往不同的區域。
  - 走完才是完整流程。
- git add 指令把檔案從工作目錄移至暫存區(或索引)。
- git commit 指令會把暫存區的內容移至儲存庫。

工作區、暫存區與儲存庫2

- 一定要二段式嗎？
  - 對於大部分情況，可以在 Commit 的時候多加一個 -a 的參數，縮短這個流程：

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git commit -a -m "commit again"
[master 983fbbb] commit again
 1 file changed, 1 insertion(+), 1 deletion(-)
```

  - Add與commit的類比：先把要回收的東西逐一放到推車上(add)，等累積到一定份量後再拿去回收場(commit)。

檢視Git紀錄1

- *git log* 指令：檢視紀錄 (誰、何時、做了甚麼)

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git log
commit 983fbbb6a4af820ddd7276121e793647a7be1d86
(HEAD -> master)
Author: Albert Ma <albert@ntou.edu.tw>
Date:   Wed Dec 18 10:53:20 2024 +0800

    commit again

commit f34a359493addd876c1c45010afbb07d8da244fc
Author: Albert Ma <albert@ntou.edu.tw>
Date:   Wed Dec 18 10:29:12 2024 +0800

    init commit
```

983fbbb6a4af820ddd7276121e793647a7be1d86是使用SHA-1 (Secure Hash Algorithm 1) 演算法所計算的結果，每個Commit都有一個這樣的值。

檢視Git紀錄2

- *git log*的結果可以更精簡

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git log --oneline --graph
* 983fbbb (HEAD -> master) commit again
* f34a359 init commit
```

刪除檔案1

- 如果把檔案自檔案總管刪除了:

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        deleted:    MyFirst.java

no changes added to commit (use "git add" and/or "git commit -a")
```

可以看到 MyFirst.java這個檔案目前的狀態是 deleted

刪除檔案2

- 接著還是一樣進行add和commit:

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git add MyFirst.java

user@DESKTOP MINGW64 ~/gittest (master)
$ git commit -m "delete MyFirst.java"
[master 01463f9] delete MyFirst.java
 1 file changed, 57 deletions(-)
 delete mode 100644 MyFirst.java

user@DESKTOP MINGW64 ~/gittest (master)
$ git status
On branch master
nothing to commit, working tree clean
```

刪除檔案3

- *git rm* 指令：讓git幫忙刪，會直接把變更送到暫存區

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git rm MyFirst.java
rm 'MyFirst.java'

user@DESKTOP MINGW64 ~/gittest (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    MyFirst.java


user@DESKTOP MINGW64 ~/gittest (master)
$ git commit -m "delete MyFirst.java"
[master 8081fa8] delete MyFirst.java
 1 file changed, 57 deletions(-)
 delete mode 100644 MyFirst.java
```

改檔案名稱

- *git mv*指令可以改名字，也會把變更送到暫存區

```
user@DESKTOP MINGW64 ~/gittest
(master)
$ git mv MyFirst.java
Player.java


user@DESKTOP MINGW64 ~/gittest (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    MyFirst.java -> Player.java
```

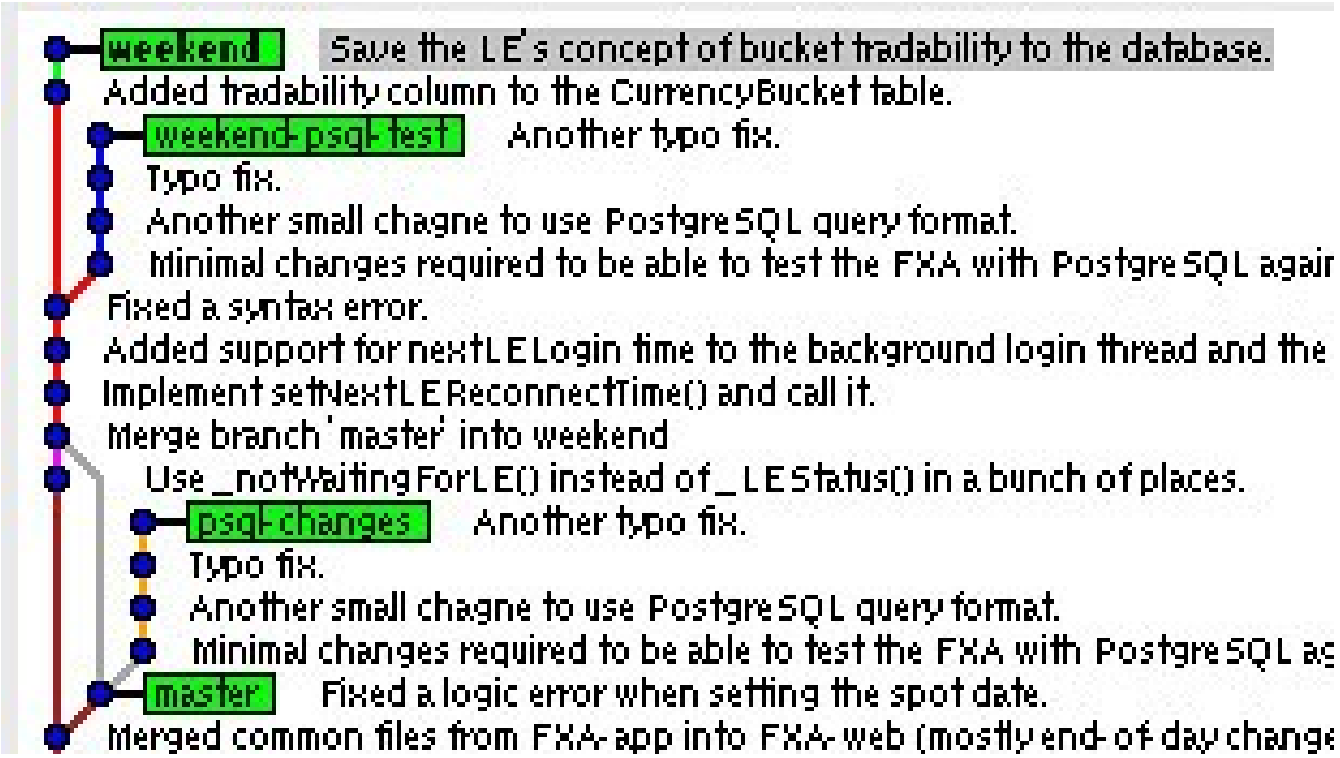
查看是誰做的

- *git blame*可以看出每一行程式是誰寫的

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git blame MyFirst.java
b5c896d7 Player.java (Albert Ma 2024-10-18 16:55:14 +0800  1)
public class Role {
b5c896d7 Player.java (Albert Ma 2024-10-18 16:55:14 +0800  2)
private String name; // it is name
b5c896d7 Player.java (Albert Ma 2024-10-18 16:55:14 +0800  3)
private int hp;
b5c896d7 Player.java (Albert Ma 2024-10-18 16:55:14 +0800  4)
private int offense;
b5c896d7 Player.java (Albert Ma 2024-10-18 16:55:14 +0800  5)
private int defense;
...
```

內建圖形化介面: gitk

- 可輸入*gitk*觀看圖形化的版本紀錄。



修改 Commit 紀錄

- 可使用 *--amend* 參數來進行 Commit，去修改commit訊息

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git commit --amend -m "intial commit again"
[master b5c896d] intial commit again
 Date: Wed Dec 18 16:55:14 2024 +0800
 1 file changed, 57 insertions(+)
 create mode 100644 Player.java

user@DESKTOP MINGW64 ~/gittest (master)
$ git log
commit b5c896d7a0295f969400bed0ff00258d486fc2bc (HEAD ->
master)
Author: Albert Ma <albert@ntou.edu.tw>
Date:   Wed Dec 18 16:55:14 2024 +0800

    intial commit again
```

若進入到vim畫面，可按[esc] 離開編輯模式，並輸入 :wq 存檔離開

回到過去的Commit

- 如果想要取消最後這次的 Commit，可使用*git revert*
  (檔案內容會回到過去)
  - 最常使用指令：*git revert HEAD --no-edit*

```
user@DESKTOP MINGW64 ~/git (master)
$ git log --oneline
e975cb8 (HEAD -> master) 3rd version
5eae413 2nd version
9702f72 1st version
```

```
user@DESKTOP MINGW64 ~/git (master)
$ git revert HEAD --no-edit
[master 2ed4f76] Revert "3rd version"
 Date: Mon May 4 13:22:19 2024 +0800
 1 file changed, 1 insertion(+), 1
deletion(-)
```



```
user@DESKTOP MINGW64 ~/git
(master)
$ git log --oneline
2ed4f76 (HEAD -> master) Revert
"3rd version"
e975cb8 3rd version
5eae413 2nd version
9702f72 1st version
```

```
user@DESKTOP MINGW64 ~/git
(master)
$ git status
On branch master
nothing to commit, working tree
clean
```

拆掉Commit

- 可用 *git reset* **拆掉**Commit (目前的檔案內容不會變)
  - 把目前的狀態設定成某個指定的 Commit 的狀態，通常適用於尚未推出去的 Commit。
  - 從目前的版本往前退一版: $ git reset HEAD^
  - 從目前的版本往前退兩版: $ git reset HEAD~2

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git log --oneline
d7e0f69 (HEAD -> master) 3rd commit
9fd93f5 2nd version
e30c139 1st version

user@DESKTOP MINGW64 ~/gittest (master)
$ git reset master^
Unstaged changes after reset:
M       MyFirst.java
```

```
user@DESKTOP MINGW64 ~/gittest
(master)
$ git log --oneline
9fd93f5 (HEAD -> master) 2nd version
e30c139 1st version


user@DESKTOP MINGW64 ~/git (master)
$ git status
On branch master
Changes not staged for commit:
…
```

為什麼要使用分支(Branch)？

- 當開始越來越多同伴一起在同一個專案工作的時候，彼此的Commit會互相影響。
- 新分支可讓與主分支暫時隔離
  - 增加新功能，或是修正 Bug，或是想實驗看看某些新的做法。
  - 待做完確認沒問題之後再合併回來，不會影響正在運行的產品線。

查看分支

- *git branch*指令可以查看目前分支
  - 一開始應該只有master

```
user@DESKTOP MINGW64 ~/gittest
(master)
$ git branch
* master
```

分支新增、改名與刪除

- *git branch*指令、以及運用*-m*與*-d*參數

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git branch hotfix

user@DESKTOP MINGW64 ~/gittest (master)
$ git branch -m hotfix feature

user@DESKTOP MINGW64 ~/gittest (master)
$ git branch
  feature
* master


user@DESKTOP MINGW64 ~/gittest (master)
$ git branch -d feature
Deleted branch feature (was b5c896d).

user@DESKTOP MINGW64 ~/gittest (master)
$ git branch
* master
```

切換分支

- *git checkout*指令可以切換分支

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git branch hotfix

user@DESKTOP MINGW64 ~/gittest (master)
$ git checkout hotfix
Switched to branch 'hotfix'
```

合併分支

- 若新分支已經新增多個commit，要併回原分支，要使用*git merge*指令

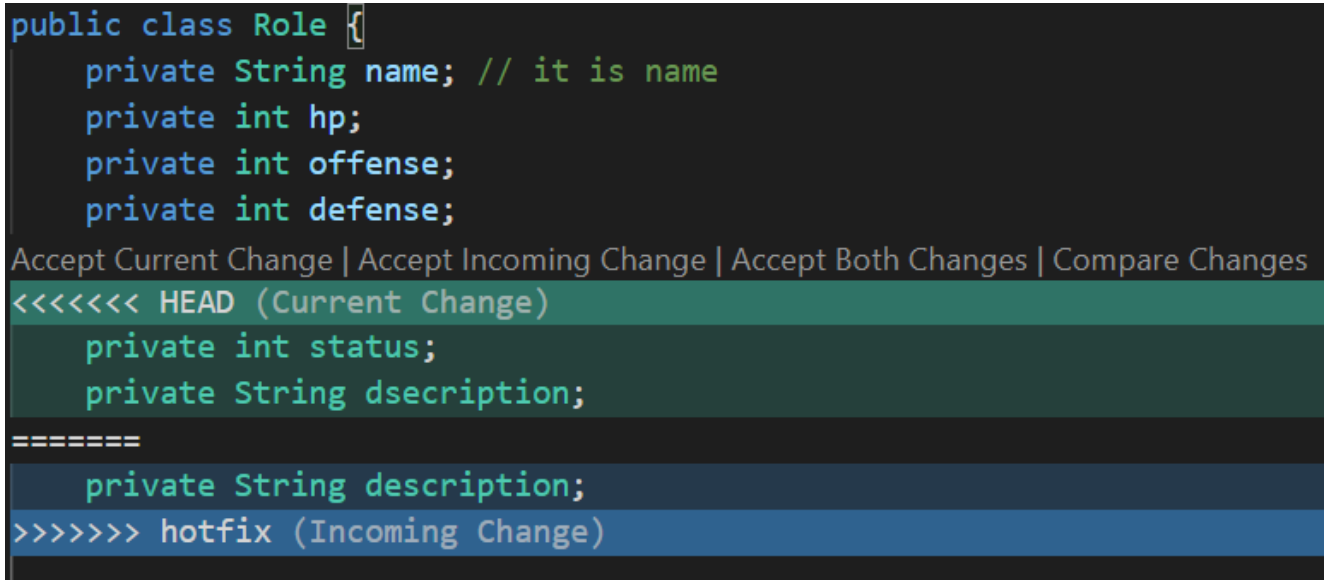
```
user@DESKTOP MINGW64 ~/gittest (hotfix)
$ git checkout master
Switched to branch 'master'

user@DESKTOP MINGW64 ~/gittest (master)
$ git merge hotfix
Updating 0defb56..4be2cad
Fast-forward
 MyFirst.java | 4 +++-
 1 file changed, 3 insertions(+), 1 deletion(-)
```

合併發生衝突該如何解決1


```
user@DESKTOP MINGW64 ~/gittest (master)
$ git merge hotfix
Auto-merging Role.java
CONFLICT (content): Merge conflict in Role.java
Automatic merge failed; fix conflicts and then commit the result.
```

衝突發生了!



Git會幫忙在程式碼中標示衝突的部分

合併發生衝突該如何解決2

- 解決衝突後(通常需要雙方討論)重新add與commit

```java
public class Role {
    private String name; // it is name
    private int hp;
    private int offense;
    private int defense;
    private int status;
    private String description;
```

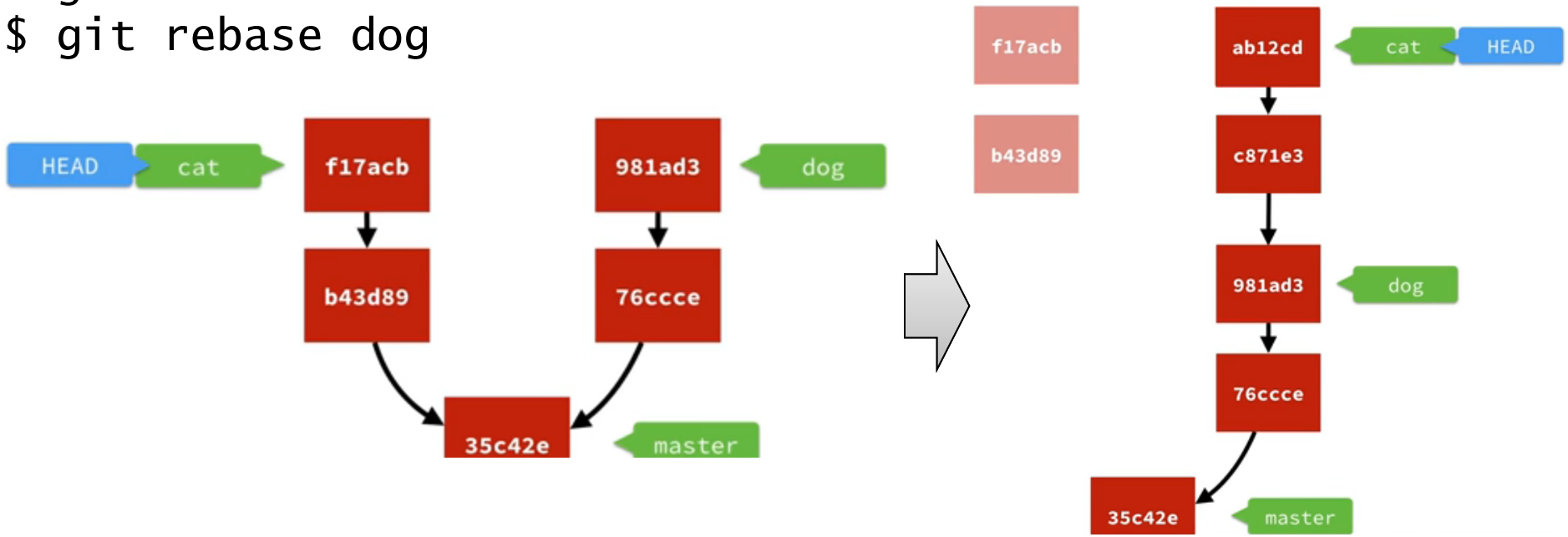
```
user@DESKTOP MINGW64 ~/gittest (master|MERGING)
$ git add *.java

user@DESKTOP MINGW64 ~/gittest (master|MERGING)
$ git commit -m "conflict solved"
[master ed43ad7] conflict solved
```

另一種合併分支方式: rebase

- git rebase dog (目前在cat分支)
  - cat分支使用 dog 分支當做我新的參考基準
  - 將cat分支接到dog之上

```
$ git checkout cat
$ git rebase dog
```



https://gitbook.tw/chapters/branch/merge-with-rebase.html

標籤(tag)是什麼？

- 在 Git，標籤(tag)是一個指向某一個 Commit 的指標。
- 通常在開發軟體有完成特定的里程碑，例如軟體版號 0.1.0 或是 beta-release ，可以使用標籤註記。
  - 可參考語意化標籤：https://semver.org/lang/zh-TW/

有附註(Annotated)的標籤

- *git tag*指令可以新增標籤
- *git show*指令可顯示某個標籤的詳細資訊

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git tag -a beta -m "it is able to be tested"
```

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git show beta
tag beta
Tagger: Albert Ma <albert@ntou.edu.tw>
Date:   Wed Dec 18 18:05:14 2024 +0800

it is able to be tested

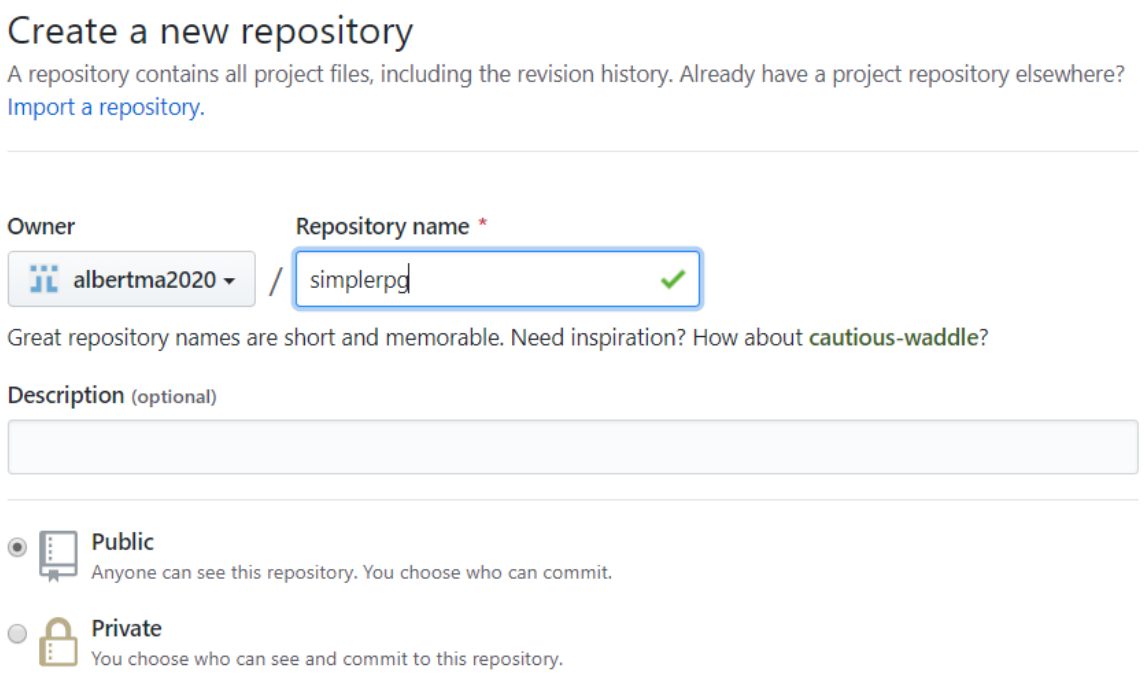
commit
ed43ad75d3109c12a0b5aae5212795aa7d375e52
(HEAD -> master, tag: beta)
Merge: 17a7970 d2285b6
Author: Albert Ma <albert@ntou.edu.tw>
Date:   Wed Dec 18 17:59:47 2024 +0800
...
```

GitHub 是什麼？

- https://github.com/
- 目前全球最大的 Git Server。
- 可以幫忙貢獻其它人的專案，其它人也可以協助您的專案。
- 是開發者最好的履歷。

創建GitHub Repository

- 請先註冊好帳號，然後選擇[Start a Project]
- 填完Repository name，按下[Create Repository]即可。

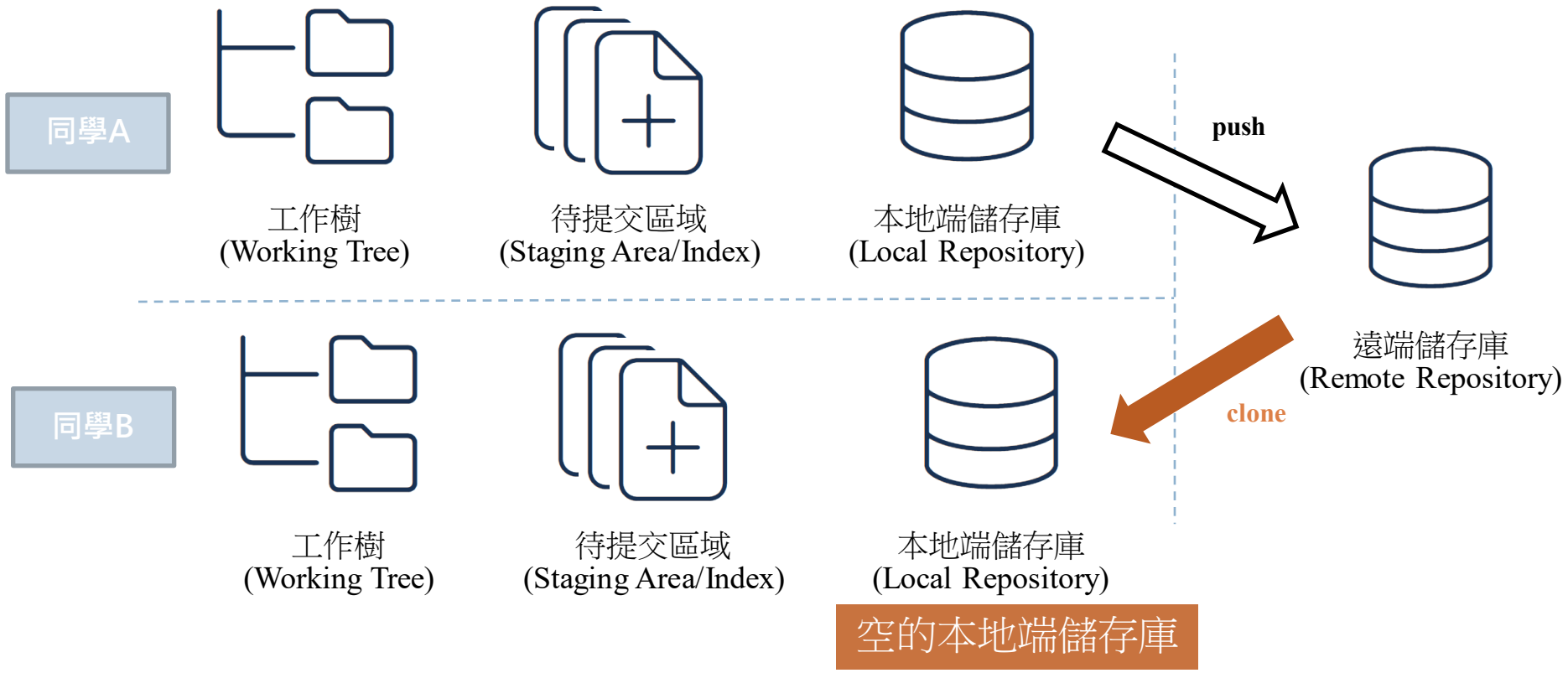


設定SSH (Secure Shell)

- Generating a new SSH key
  - Open Git Bash
    - *$ ssh-keygen -t ed25519 -C your_email@example.com*
- Copying the SSH public key to your clipboard
  - 使用者目錄\.ssh，以編輯器打開.pub檔，再複製全部內容
- Adding a new SSH key to your GitHub account
  - https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account

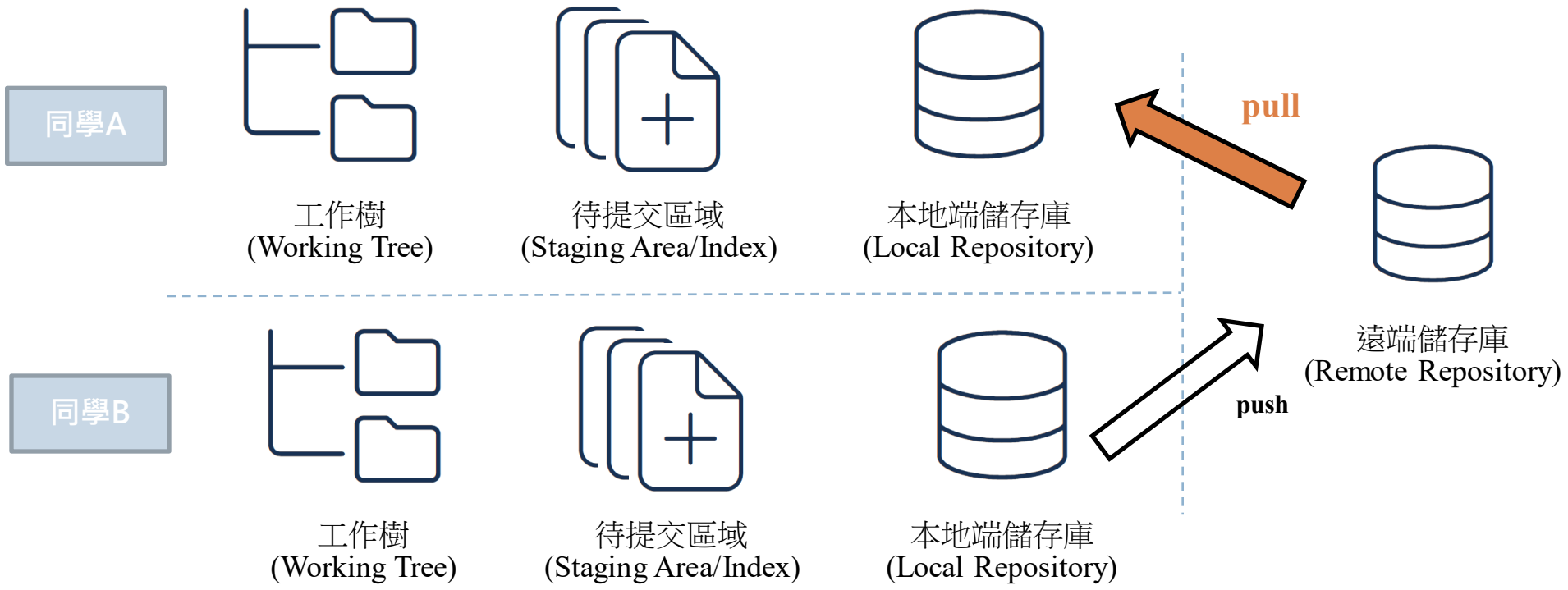
GitHub 多人合作 – Collaborators1

- 協同合作者可將遠端儲存庫(擁有者所維護)的提交歷程複製(clone)到本地端儲存庫



GitHub 多人合作 – Collaborators2

- 協同合作者可以將修改的提交透過push指令同步至遠端儲存庫
- 而其他團隊成員可以透過pull指令將遠端儲存庫的最新提交版本同步至本地端儲存庫



Git Push1

- *git remote*指令

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git remote add origin
https://GitHub.com/albertma2024/gittest.git
```

(用HTTPS、帳密登入GitHUb，目前已被禁止)

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git remote add origin
git@github.com:albertma2024/gittest
```

(用SSH的方式連結GitHub)

若需修改網址或名稱：https://backlog.com/git-tutorial/tw/reference/remote.html

Git Push1

- *git push*指令

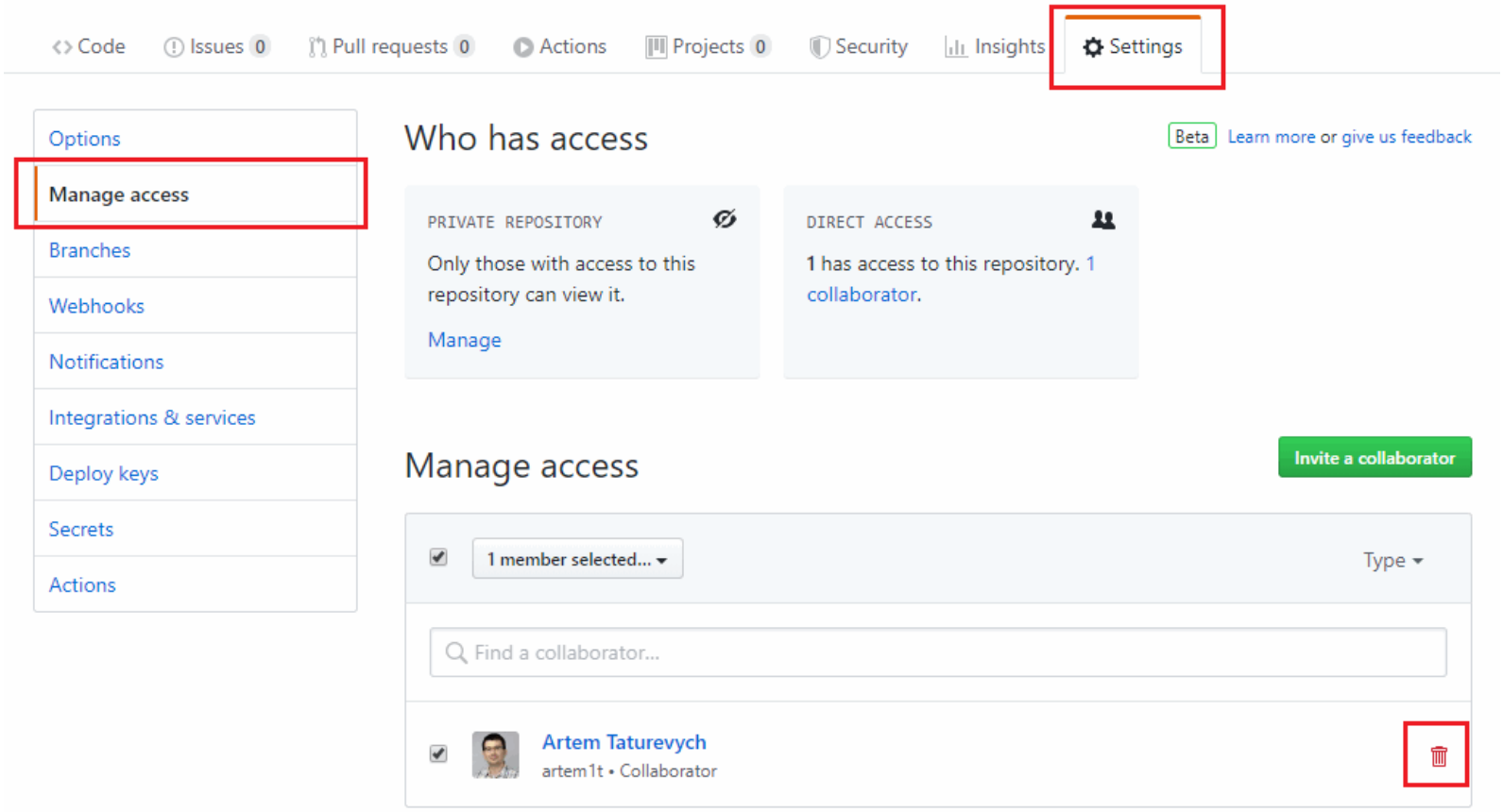
```
user@DESKTOP MINGW64 ~/gittest (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 223 bytes | 111.00
KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To https://GitHub.com/albertma2024/gittest.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch
'master' from 'origin'.
```

Git Push2

- git remote add origin
  - add 指令是指要加入一個遠端節點 (若改成*set-url*即可修改既有的設定)
  - origin是遠端節點位置的代號
- git push -u origin master
  - 把master這個分支的內容，推向origin的位置。
  - 在origin遠端 Server 上，如果master分支不存在，就建立一個。
  - 但如果本來Server上就有master分支，便會讓master分支更新到最新的進度上。
  - -u參數：
    - 它會指向並追蹤某個分支。
    - 通常 upstream 會是遠端 Server 上的某個分支。
    - 當下回執行 git push 指令而不加任何參數時，它就會預設推往 origin ，並且把 master 這個分支推上去。

加入協作者

- 你可以於”Settings”->“Manage access” 加入其他開發者，才能協同開發。



https://www.codestack.net/hosting/source-code/github/github-remove-collaborator.png

Git Clone

- 另一位開發者可以先把專案*git clone*下來。(第一次拉專案請用*clone*，而非*pull*)

```
albert@Albert-Office MINGW64 ~/workspace
$ git clone
https://GitHub.com/albertma2024/gittest.git
Cloning into 'gittest'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 0), reused 6 (delta 0),
pack-reused 0
Unpacking objects: 100% (6/6), done.
```

- 接著一樣可以*git add*、*git commit*、*git push*
  - *若要用SSH進行push，記得改remote網址*

Git Pull

- *git pull* = *git fetch* + *git merge*
  - Pull 指令會上線抓東西下來(Fetch)，並且更新本機的進度(Merge)。(發生在多人開發的情況下)

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), done.
From https://GitHub.com/albertma2024/gittest
   8215dc9..6454750  master     -> origin/master
Updating 8215dc9..6454750
Fast-forward
 test.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
```

為何有時無法Push?[1]

- 通常這個狀況會發生在多人一起開發的時候
  - 別人在你之前Push新的Commit上去了。

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git push
To https://GitHub.com/albertma2024/gittest.git
 ! [rejected]        master -> master (fetch first)
error: failed to push some refs to
'https://GitHub.com/albertma2024/gittest.git'
hint: Updates were rejected because the remote contains work
that you do
hint: not have locally. This is usually caused by another
repository pushing
hint: to the same ref. You may want to first integrate the
remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --
help' for details.
```

為何有時無法Push?2

- 你必須先Pull再Push：

```
user@DESKTOP MINGW64 ~/gittest (master)
$ git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), done.
From https://GitHub.com/albertma2024/gittest2
   908075f..8f8d69b  master     -> origin/master
Merge made by the 'recursive' strategy.
 GameGUI.java | 135 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 135 insertions(+)
 create mode 100644 GameGUI.java

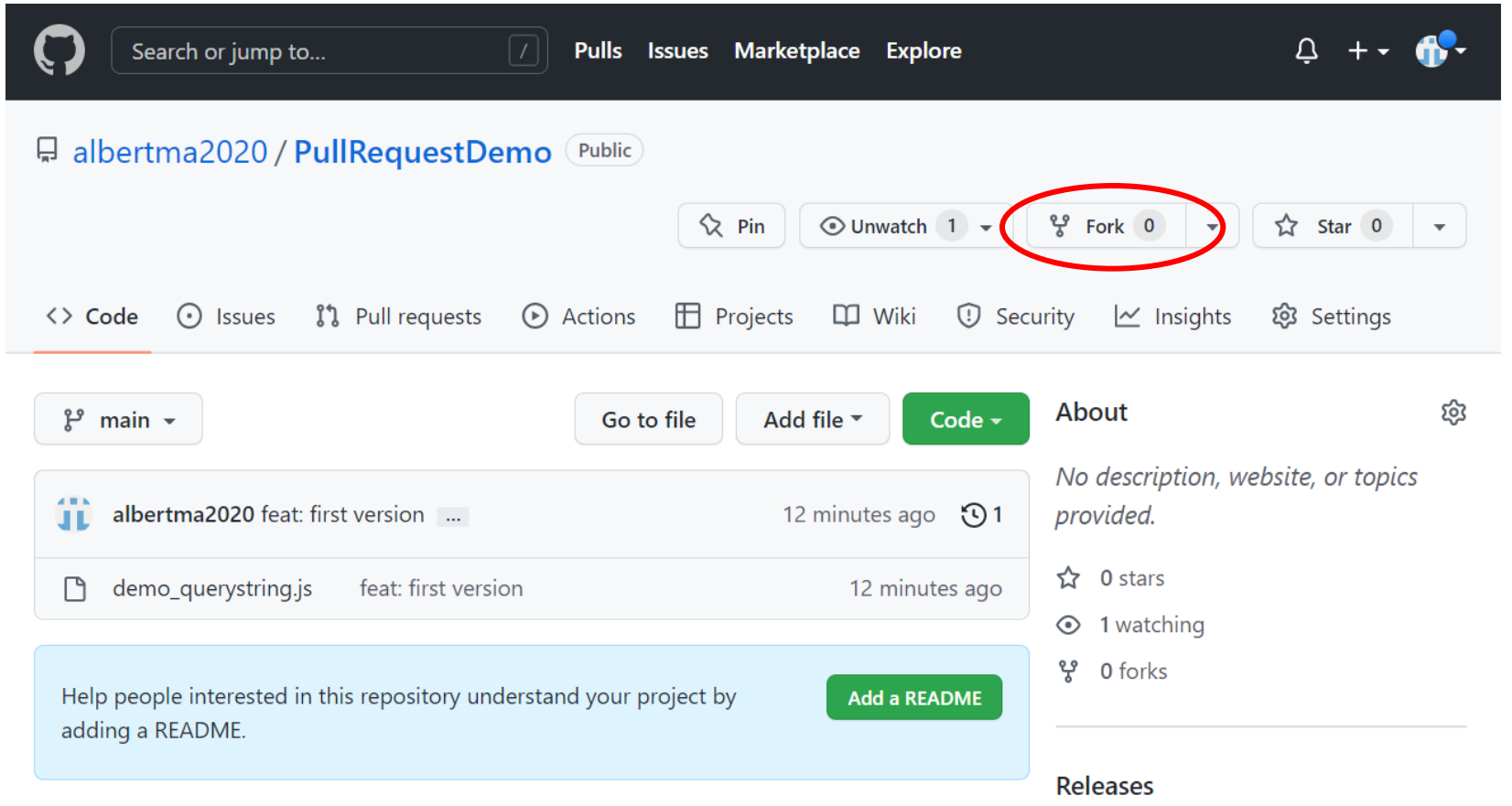
user@DESKTOP MINGW64 ~/gittest (master)
$ git push
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 1.31 KiB | 267.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0)
remote: Resolving deltas: 100% (1/1), done.
To https://GitHub.com/albertma2024/gittest2.git
   8f8d69b..bec4b5c  master -> master
```

Pull Request (PR)

- 如何提供貢獻到GitHub開源專案？
  - 先以分岔(Fork)指令複製原始的專案到自己的 GitHub 帳號底下。
  - 對複製回來的專案進行修改(你會有完整的權限)與Commit，並記得Push。
  - 接下來發Pull Request申請，讓原專案作者知道你有新開發版本。
  - 原作者核可此申請，接著他就會你做的這些新版本修改合併(Merge)到原始的專案裡。
- 一般團隊開發亦可採用此模式。

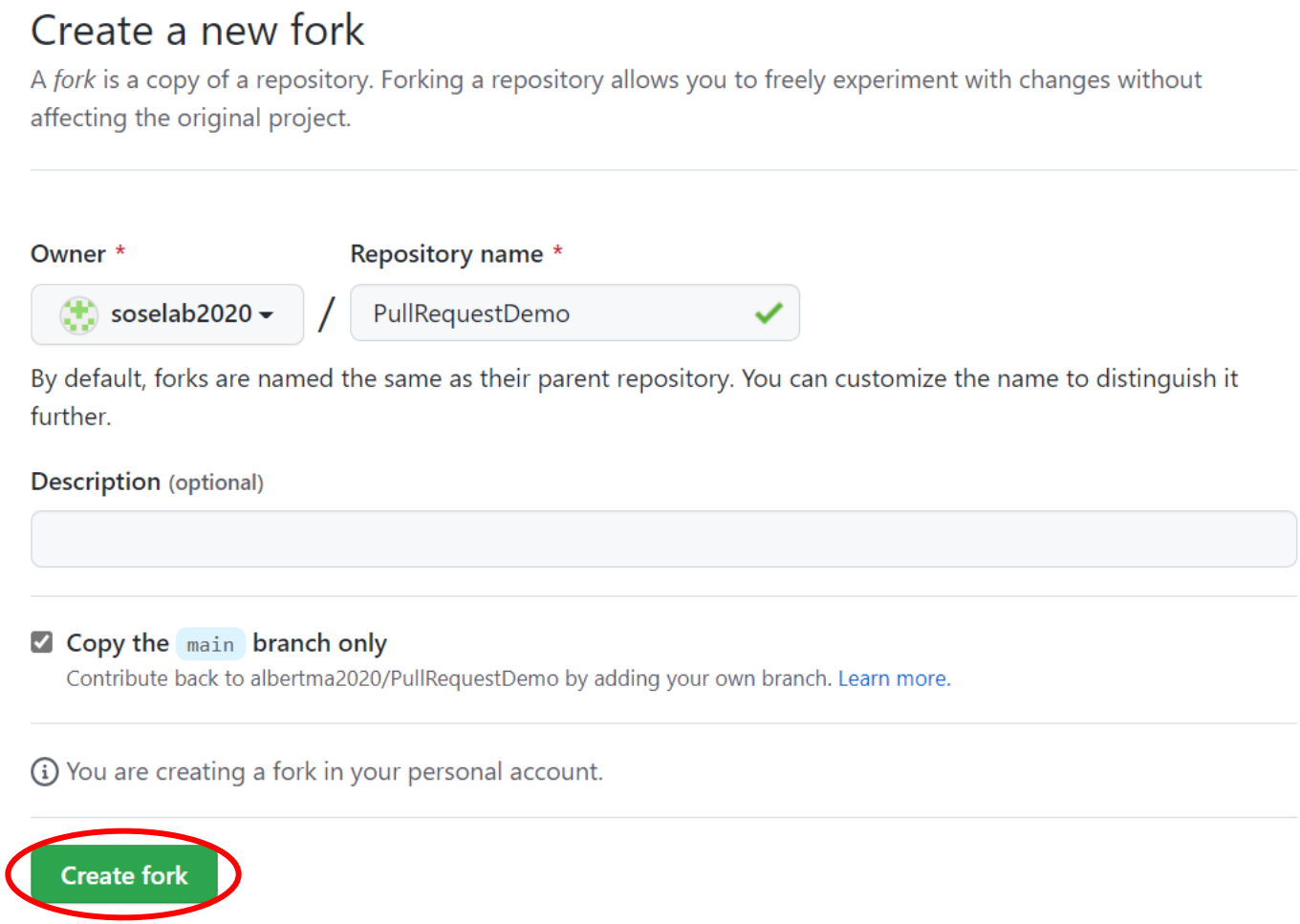
GitHub Fork1

- 貢獻者連結原始專案，選擇複製(Fork)專案



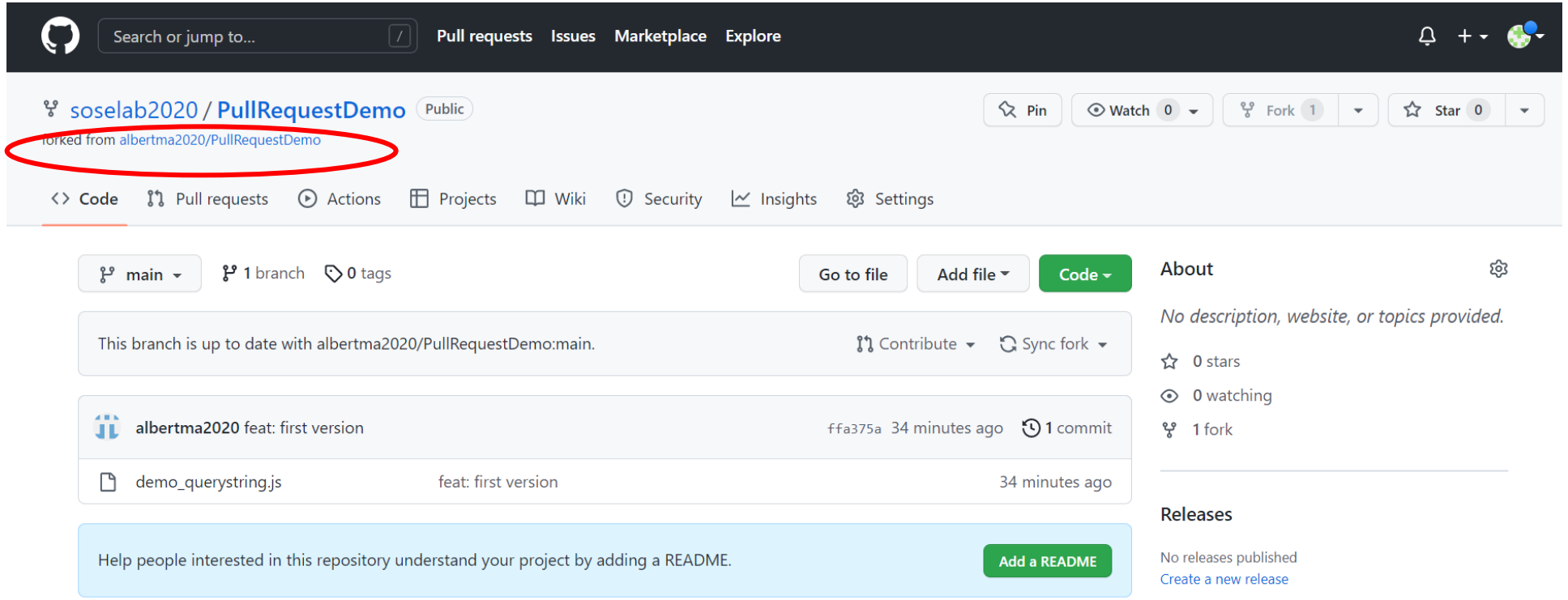
GitHub Fork2

- 執行Create fork



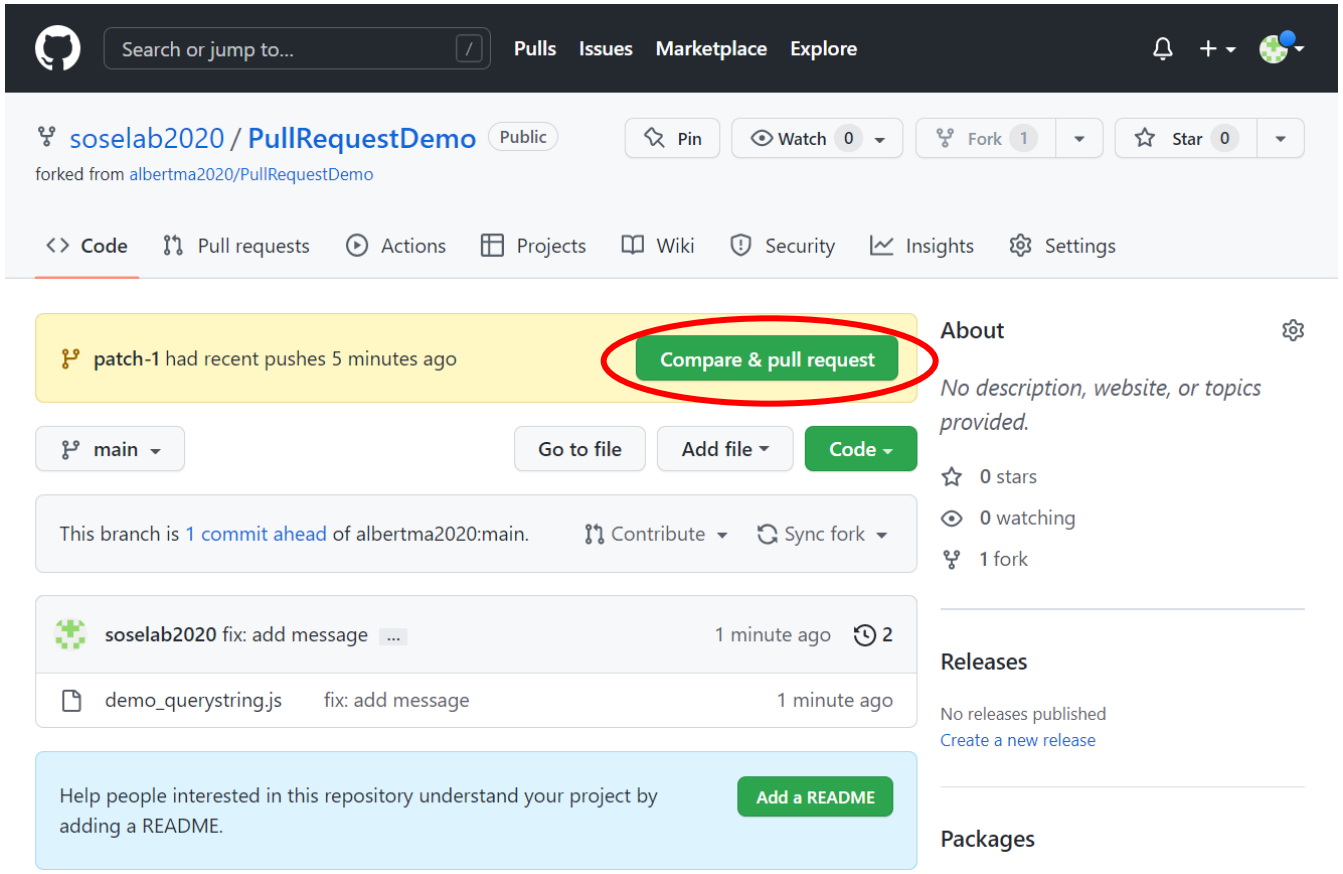
GitHub Fork3

- Fork創建成功後，即可再進行後續開發



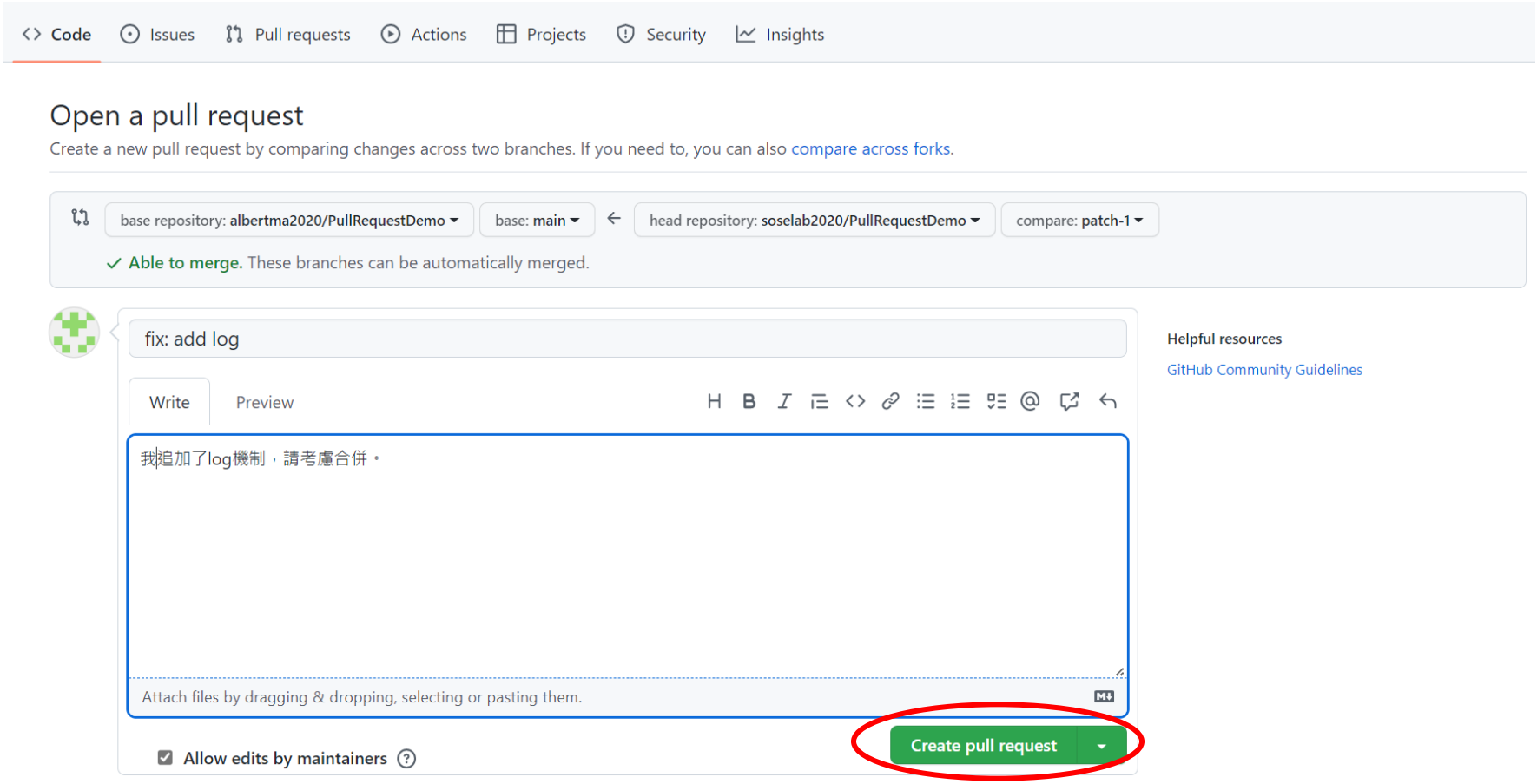
Open Pull Request1

- 貢獻者於Fork專案追加Commit後，可申請Compare&Pull Request



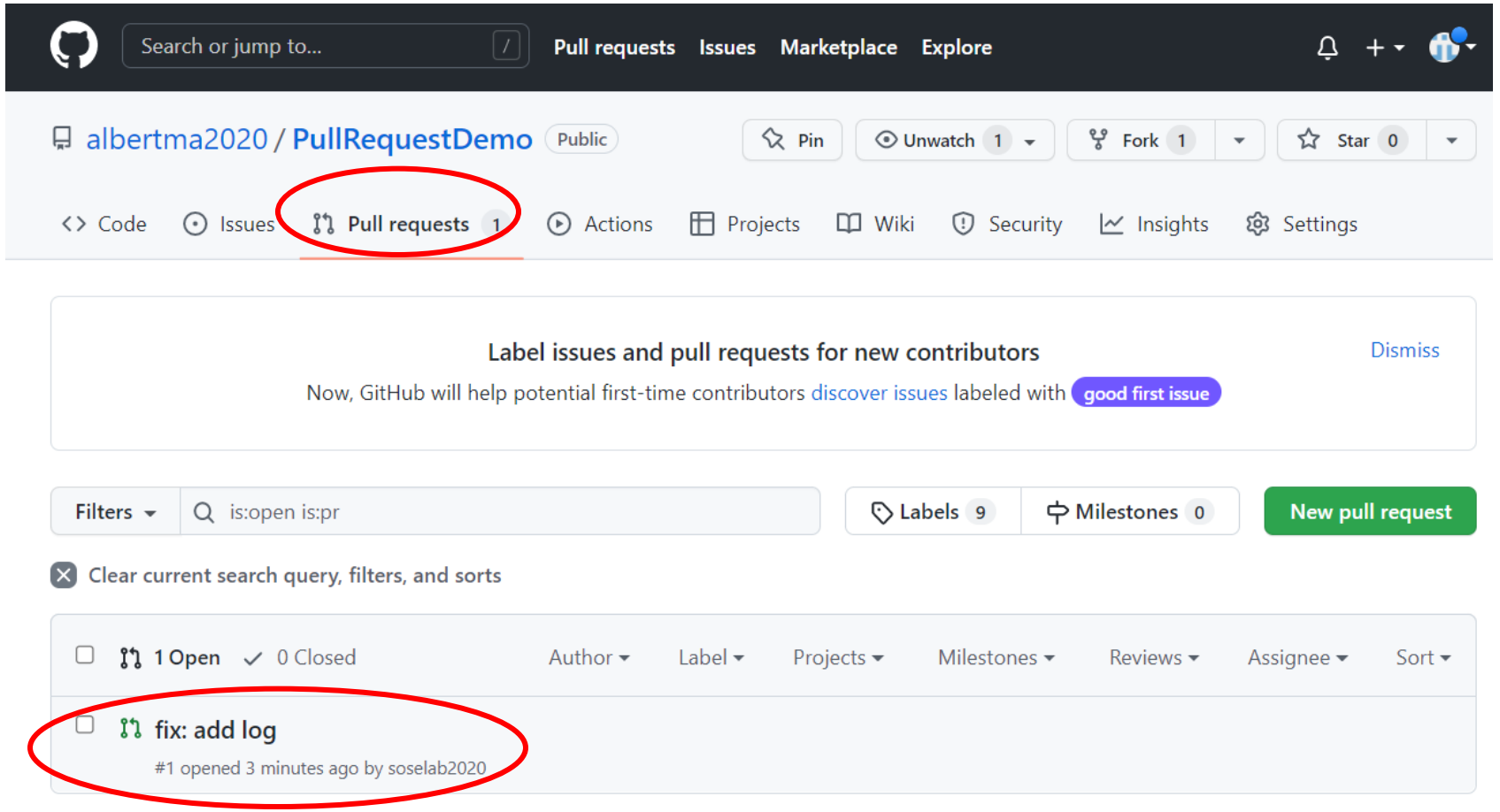
Open Pull Request2

- 貢獻者可填入相關說明，實際建立Pull Request。



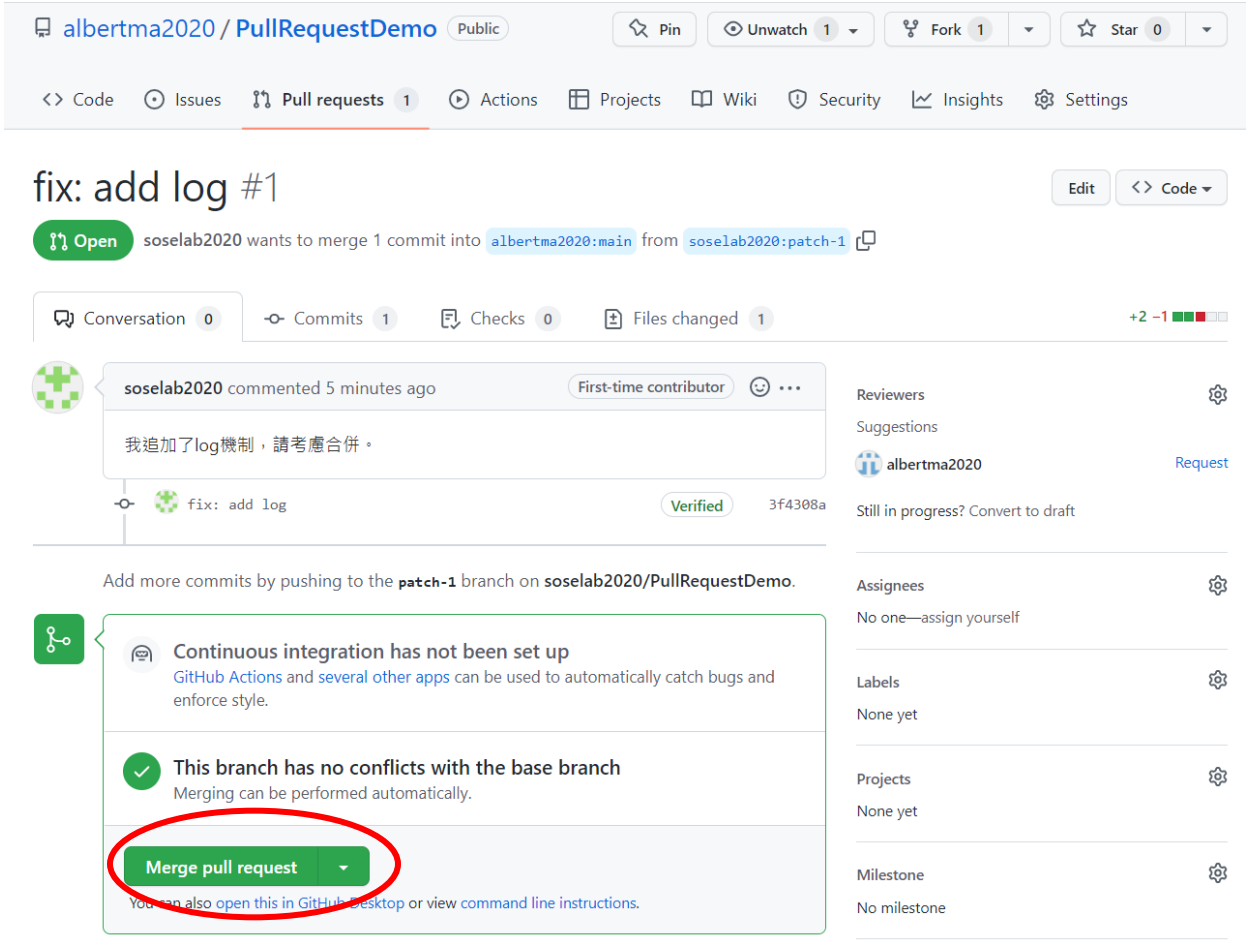
Merge Pull Request1

- 原作者可看到貢獻者提出之Pull Request。



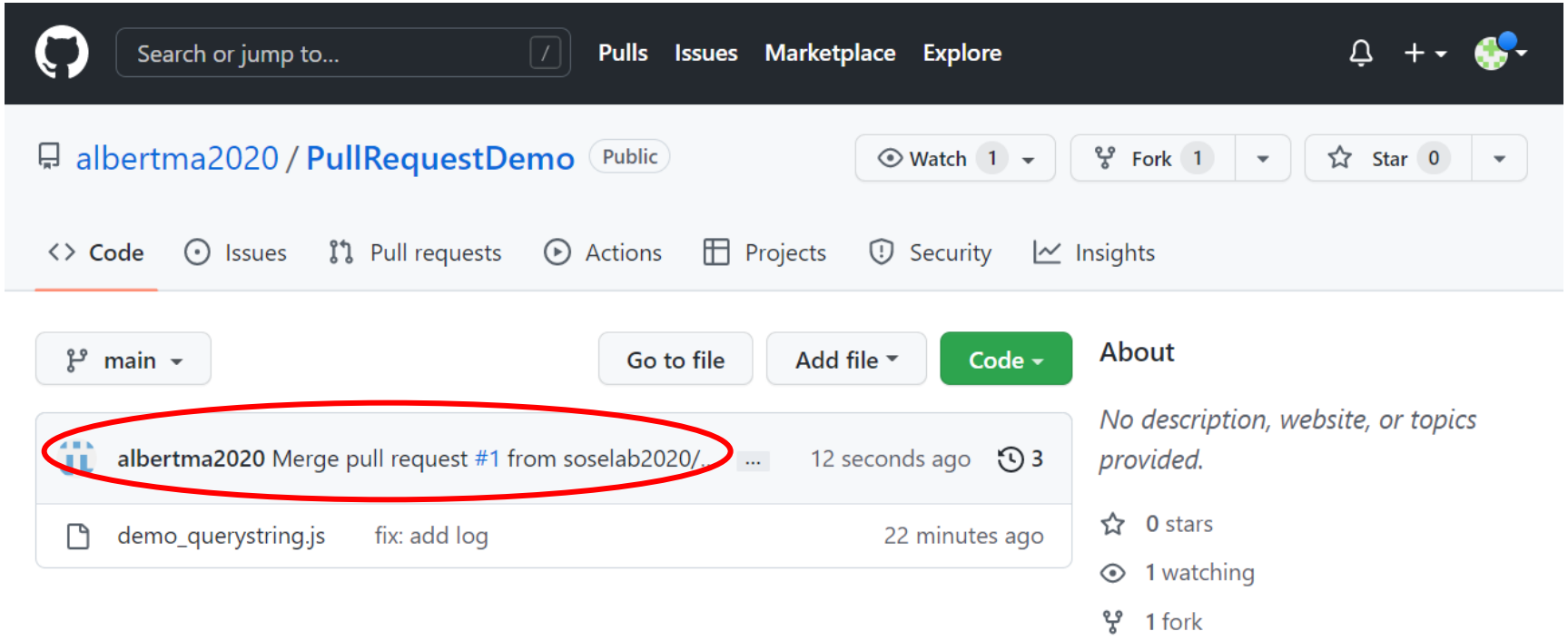
Merge Pull Request2

- 原作者可選擇合併Pull Request。



Merge Pull Request3

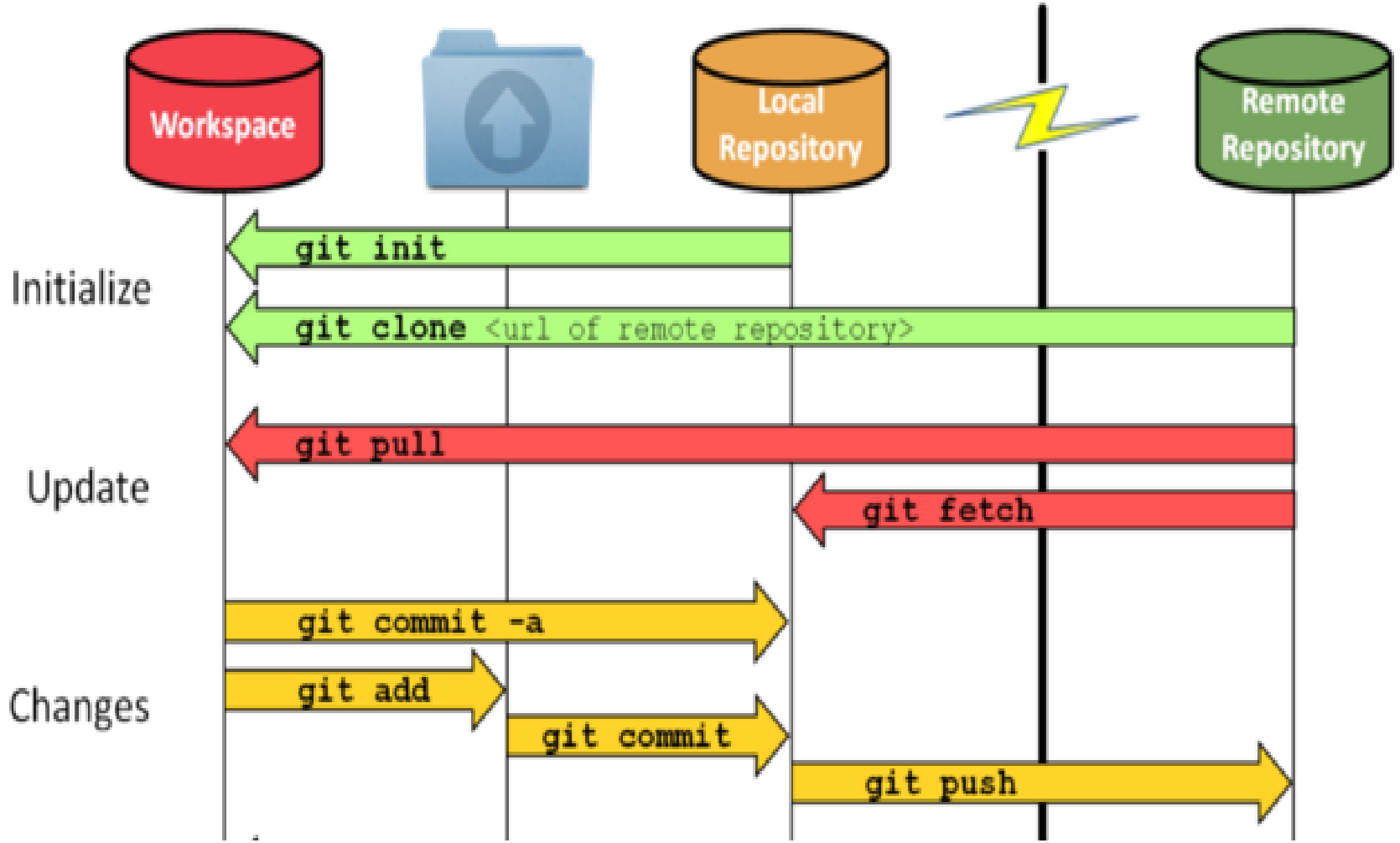
- 最後原作者即可看到Pull Request內容已被成功合併至主專案。



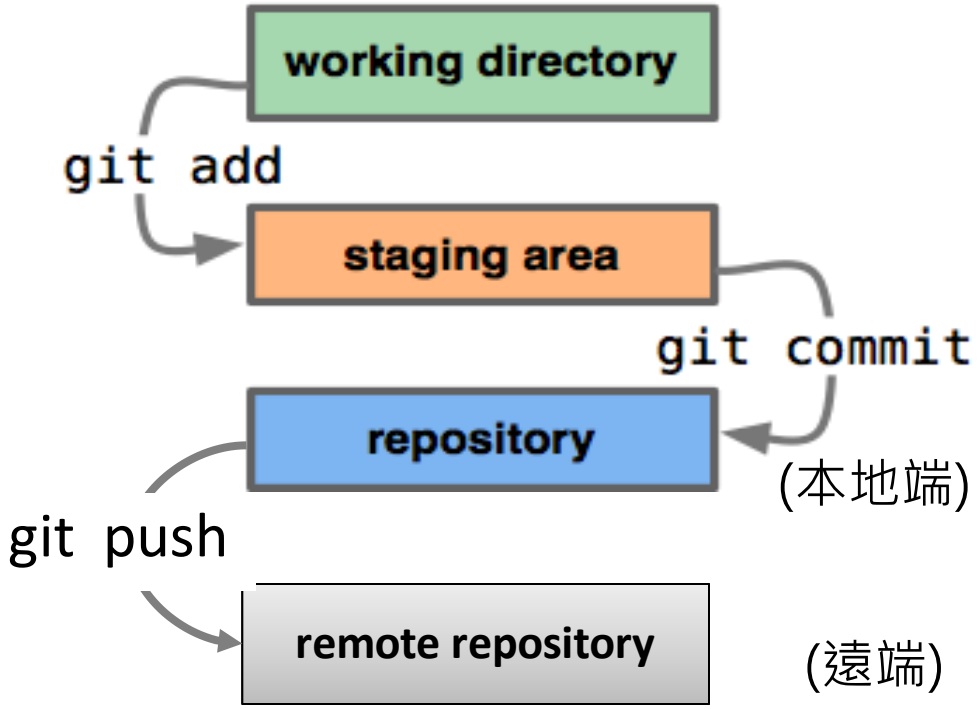
Git Ignore

- 專案目錄下若有些檔案不想被Git控管，只要在專案目錄裡放一個 .gitignore 檔案，並且設定想要忽略的規則即可。
- 常用設定規則：
  - 忽略專案各目錄中的X.txt檔案：X.txt
  - 忽略整個Y目錄: Y
  - 只忽略Y目錄中的X.txt檔案：Y/X.txt
  - 忽略所有目錄中附檔名是.txt 的檔案：*.txt
  - 忽略Y目錄中所有附檔名是.txt 的檔案：Y/*.txt

Git整體運作流程與重要指令

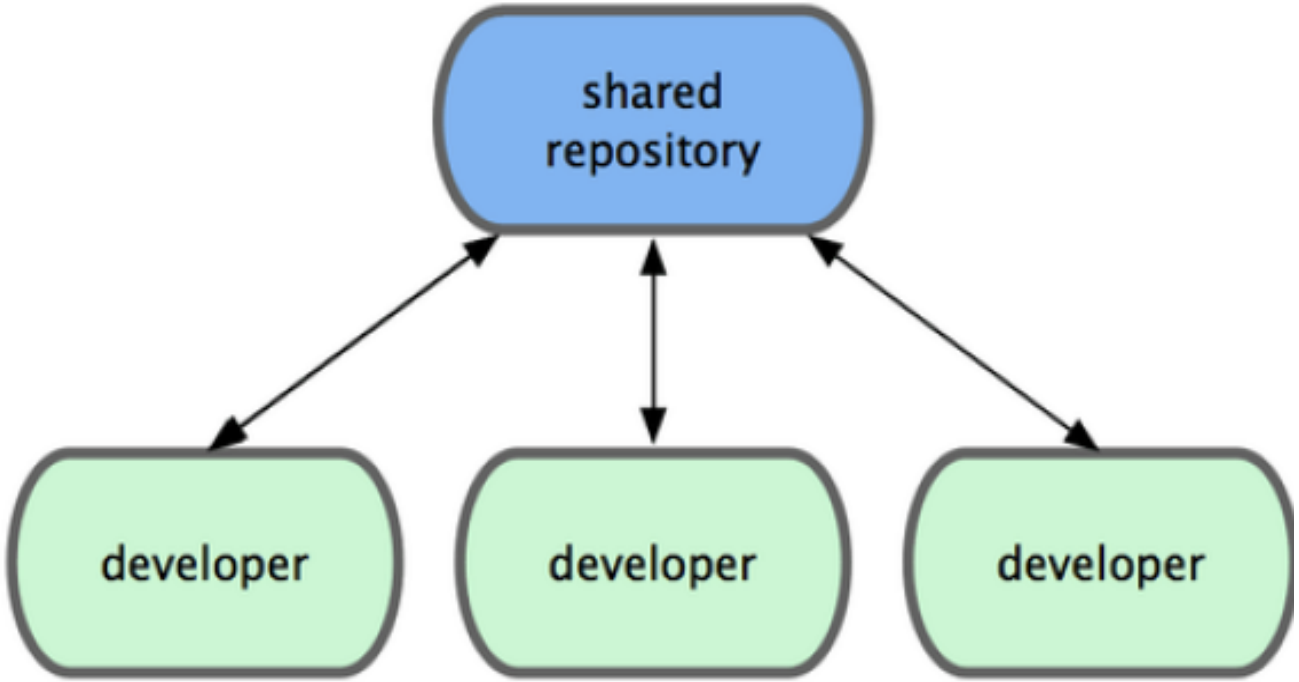


Git & GitHub Overview



https://stackoverflow.com/questions/676450/eclipse-git-what-does-staged-mean

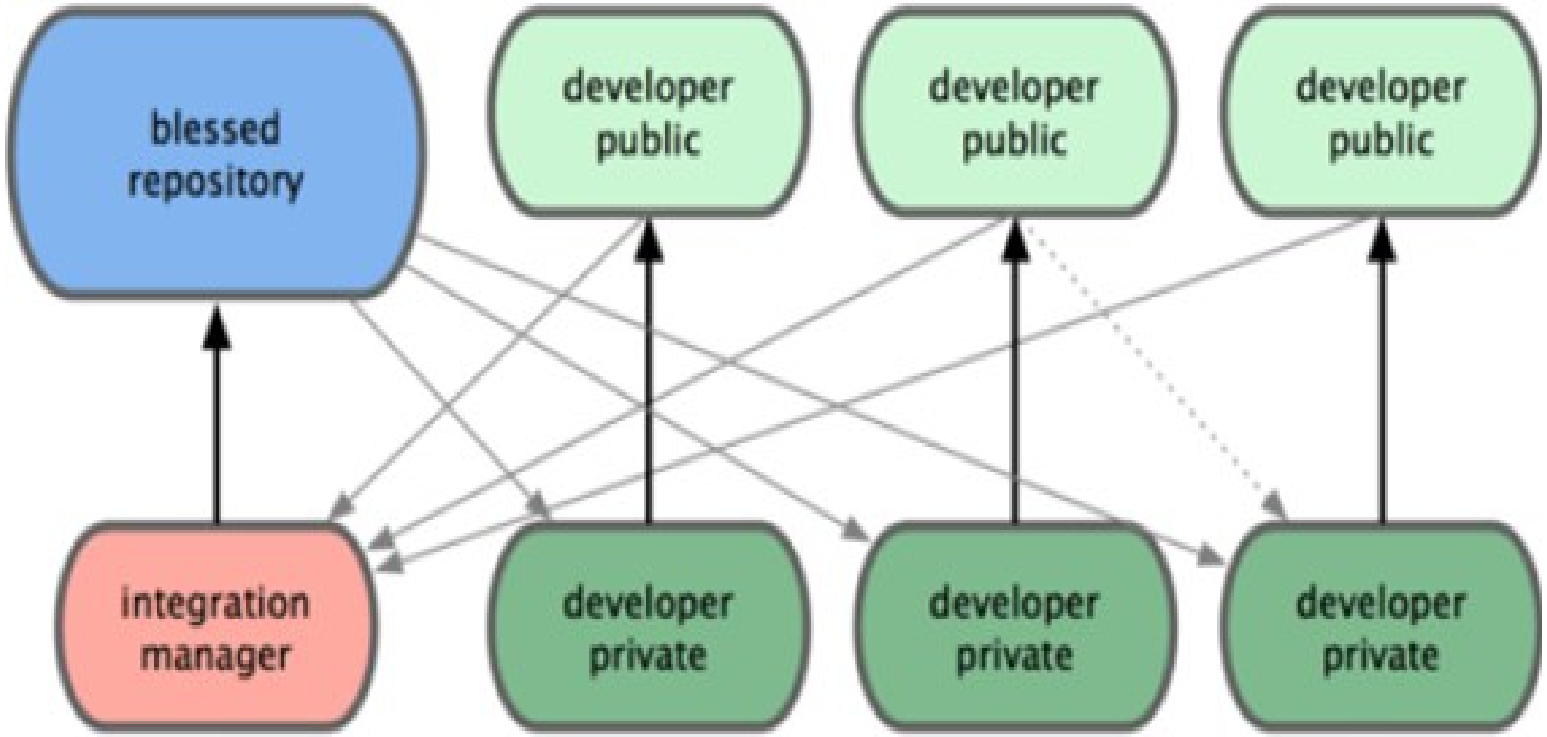
集中式版本控制[1]



集中式版本控制2

- 共用儲存庫
- 流程容易理解
- 類似 SVN 流程
- 多人同時進行版控，不同開發習慣的成員間容易產生衝突

整合管理版本控制[1]

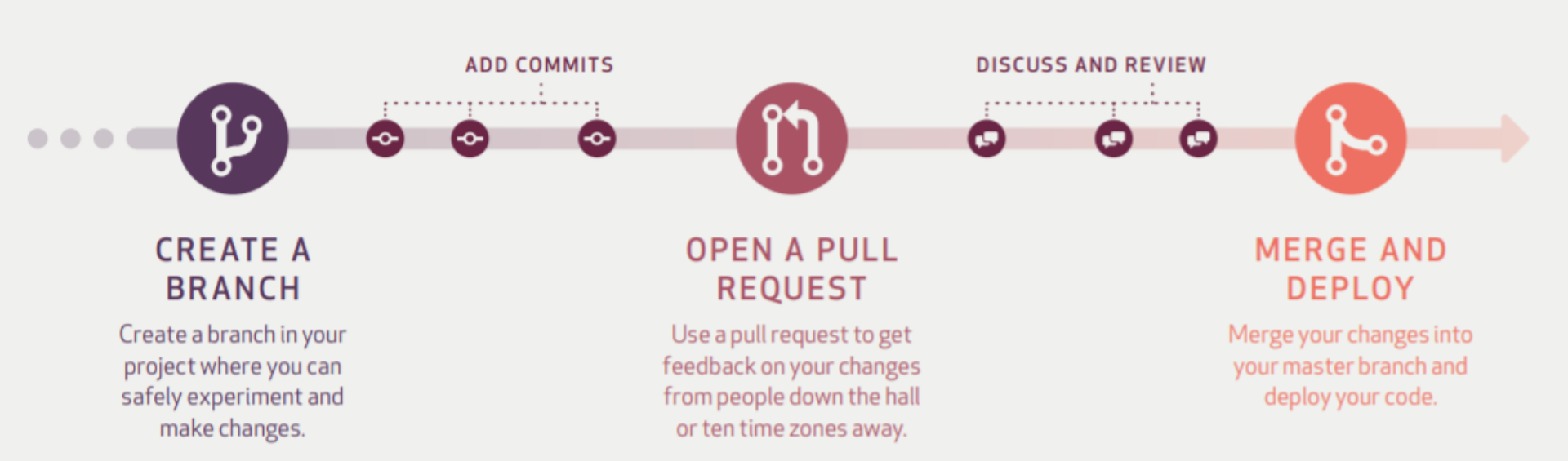


blessed repository: single source of truth (官方資料的單一來源)

整合管理版本控制2

- 專案維護人員先推送一個版本到主要儲存庫
- 專案開發成員各自複製(Fork and Clone)該儲存庫回去開發
- 專案開發成員推送變更到自己的儲存庫
- 專案開發成員向專案維護人員提出要合併的請求(Pull Request)
- 專案維護人員進行審核，再將變更進行合併整合
- 專案維護人員將合併的變更推送回主要儲存庫

GitHub Flow[1]



https://docs.github.com/en/get-started/quickstart/github-flow

GitHub Flow2

- 負責特定Feature/Fix之成員建立Branch
- 成員修改程式碼
- 成員建立Pull Request
- 其他成員進行審查
- 主責Merge的成員核可Pull Request (併到main分支)
- 建立分支的成員將其分支刪除

其他的GIT雲端服務

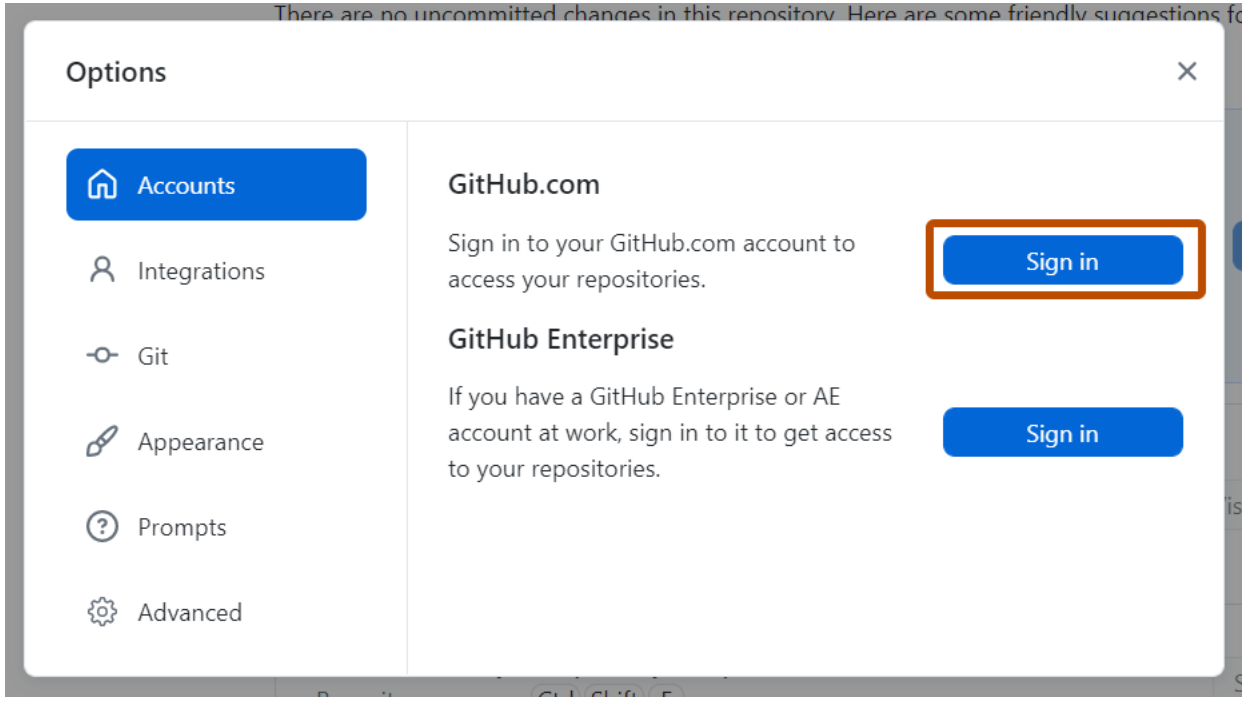
- GitLab: https://about.gitlab.com/
- BitBucket: https://bitbucket.org/

Git/GitHub圖形化介面

- GitHub Desktop: https://desktop.github.com/
- SourceTree: https://sourcetreeapp.com
- TortoiseGit: https://tortoisegit.org/

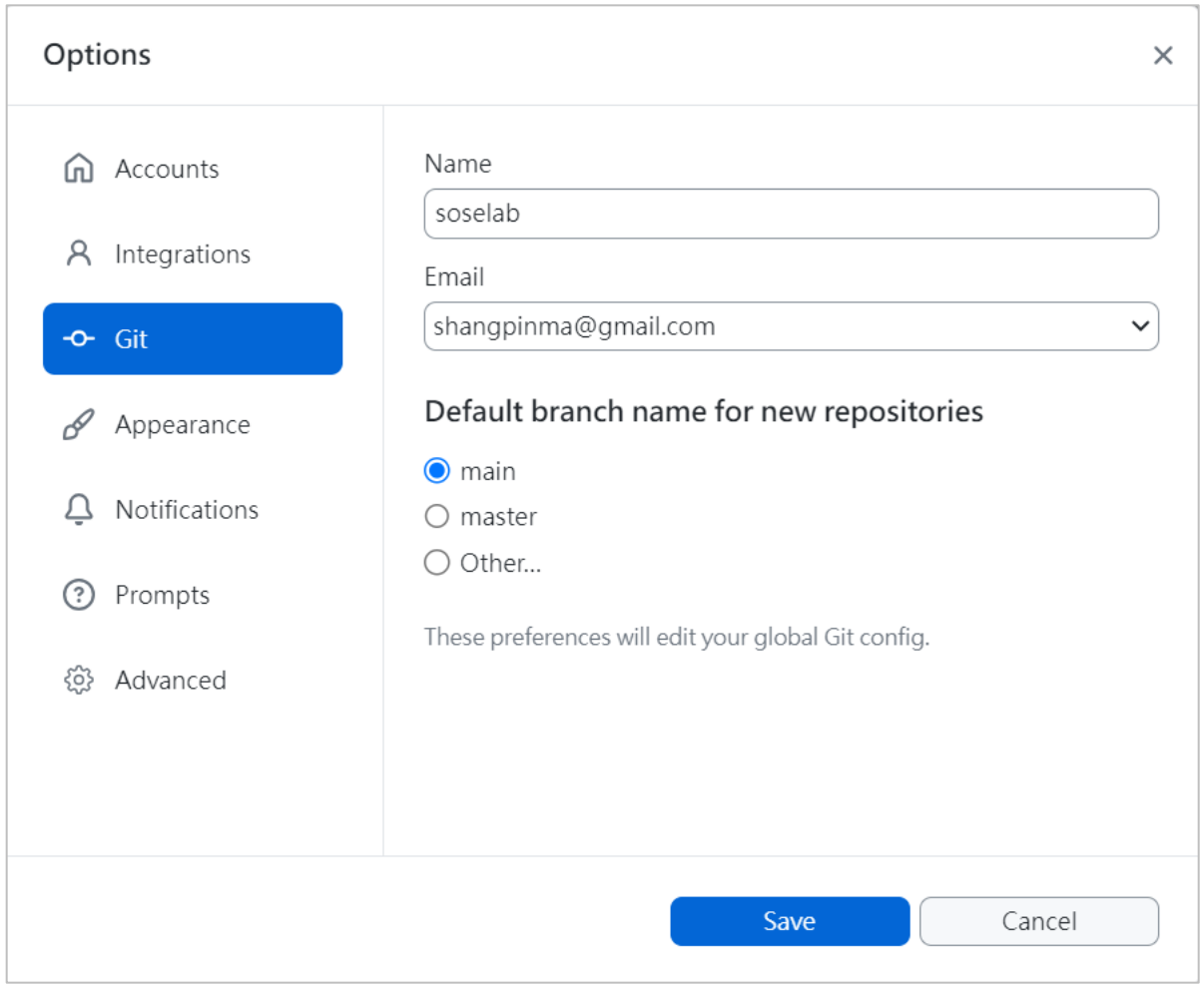
GitHub Desktop使用者帳號設定

- 安裝 GitHub Desktop 後，可以使用 GitHub 帳號對應用程式進行驗證。
  - 透過身份驗證，你可以連接到 GitHub 上的遠端儲存庫。
- 操作方式：[File]->[Options]->[Accounts]，並透過GitHub帳號登入。
  - 如果還沒有GitHub帳號，請立即申請：https://github.com/ 。



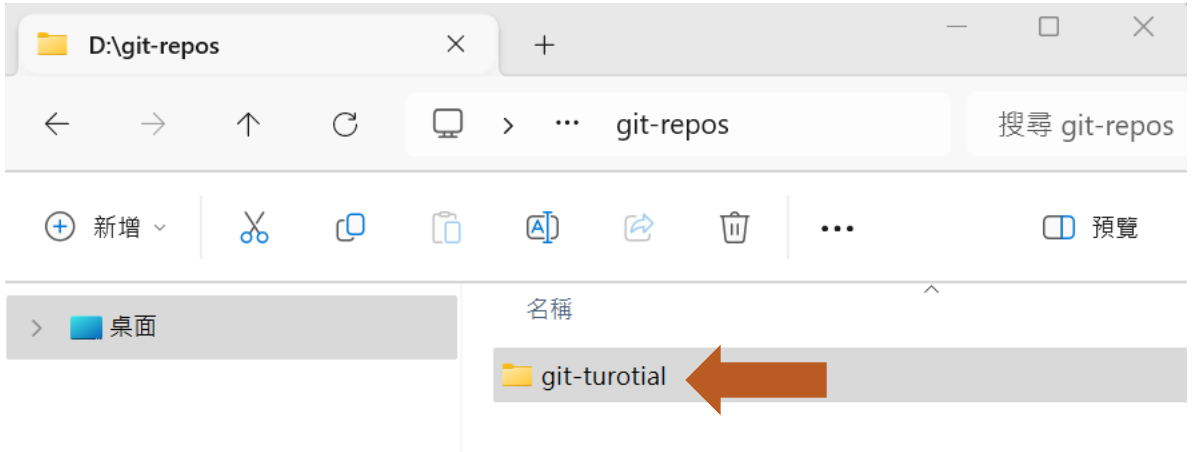
GitHub Desktop使用者帳號設定

- 在"Git"頁籤的設定部分，請設定與GitHub帳號相同的Email，以確保之後的commit之作者資訊正確。
- 此外，預設分支名稱請選擇"main"，讓Git與GitHub都具有相同的預設分支名稱。
  - 同學們可於此門課程後再去了解分支的運用。



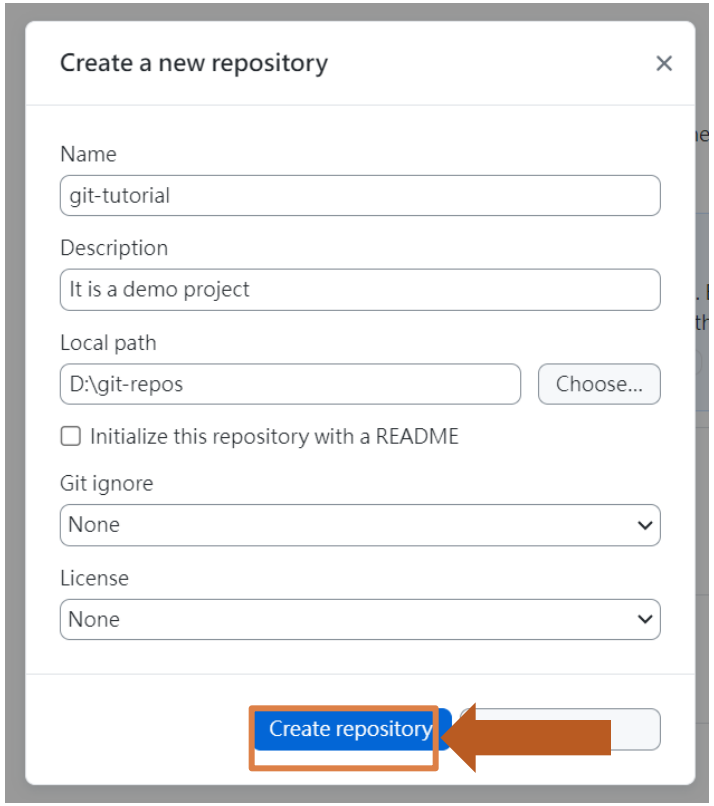
建立本地端儲存庫(Local Repository)

- 開啟檔案總管，建立版控專案**根**目錄
  - 用以管理所有的Git專案
  - 例如：目錄 *git-repos*
- 接著建立版控專案目錄(Working Tree)
  - 例如：目錄 *git-tutorial*



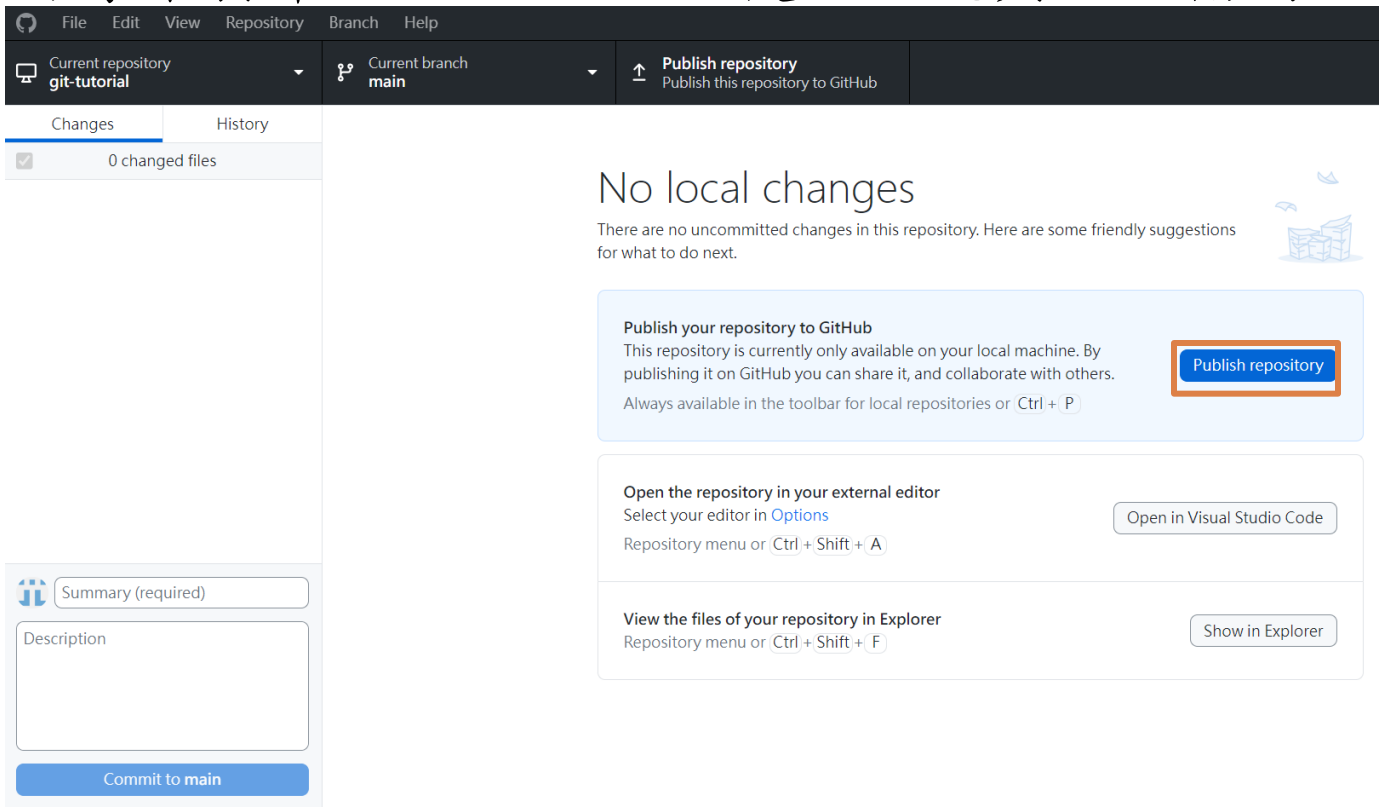
建立本地端儲存庫(Local Repository)

- 透過GitHub Desktop設定版控工作目錄(working tree)
  - [File]->[New Repository]

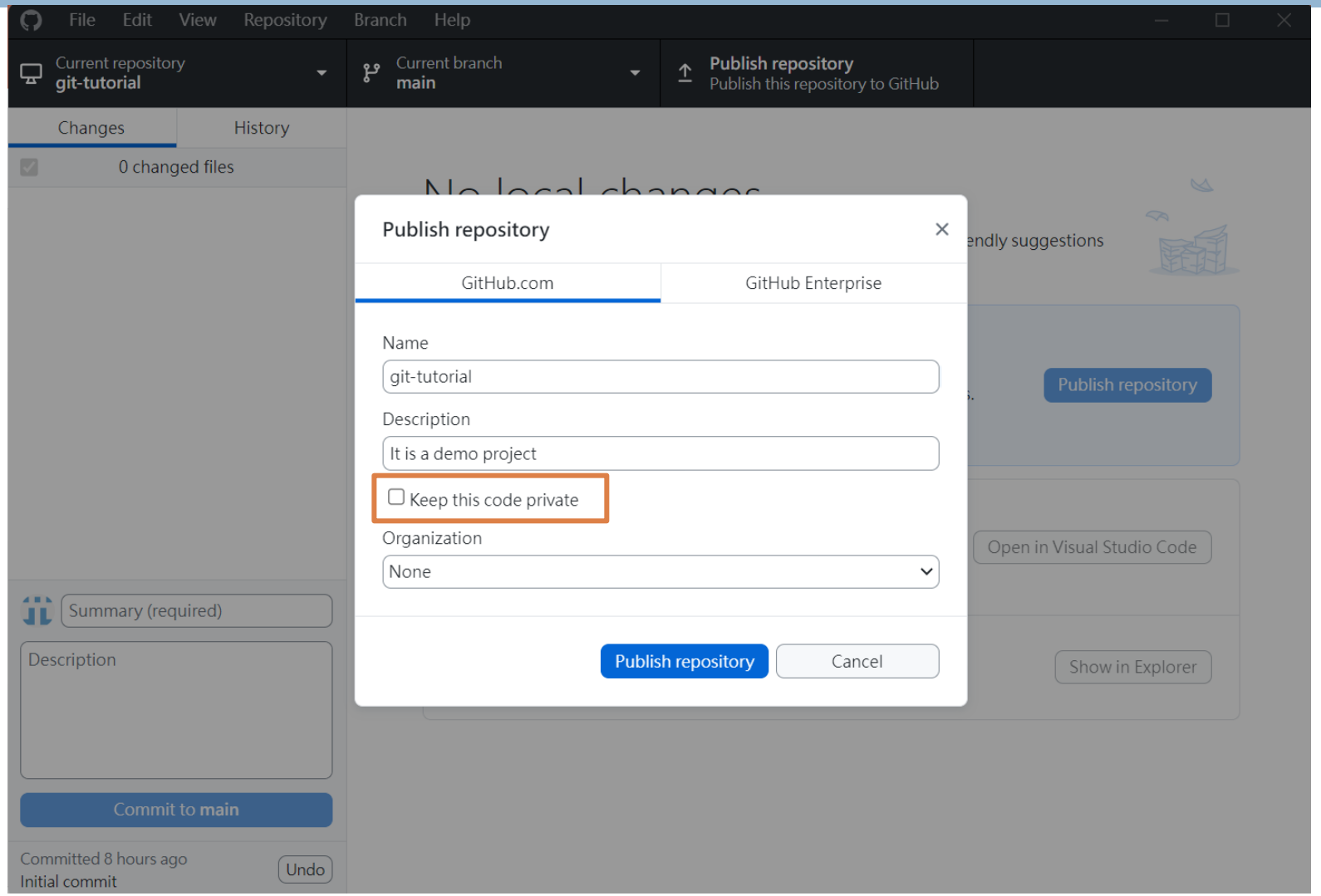


建立與連結遠端儲存庫(Remote Repository)1

- 我們可直接將本地端儲存庫與GitHub遠端儲存庫串連起來
  - 點擊"Publish Repository"功能即可完成
  - 等同將專案發布於Internet，日後可方便與他人協同合作

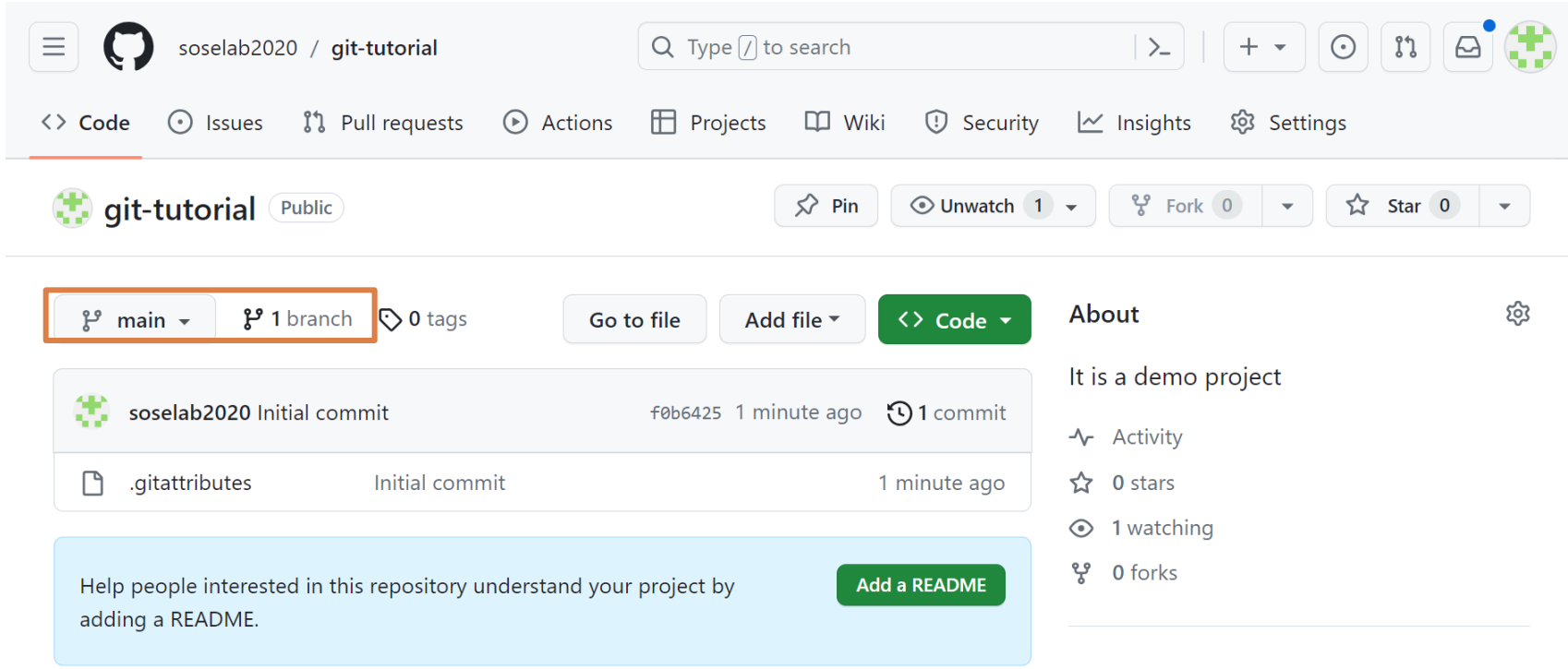


建立與連結遠端儲存庫(Remote Repository)2



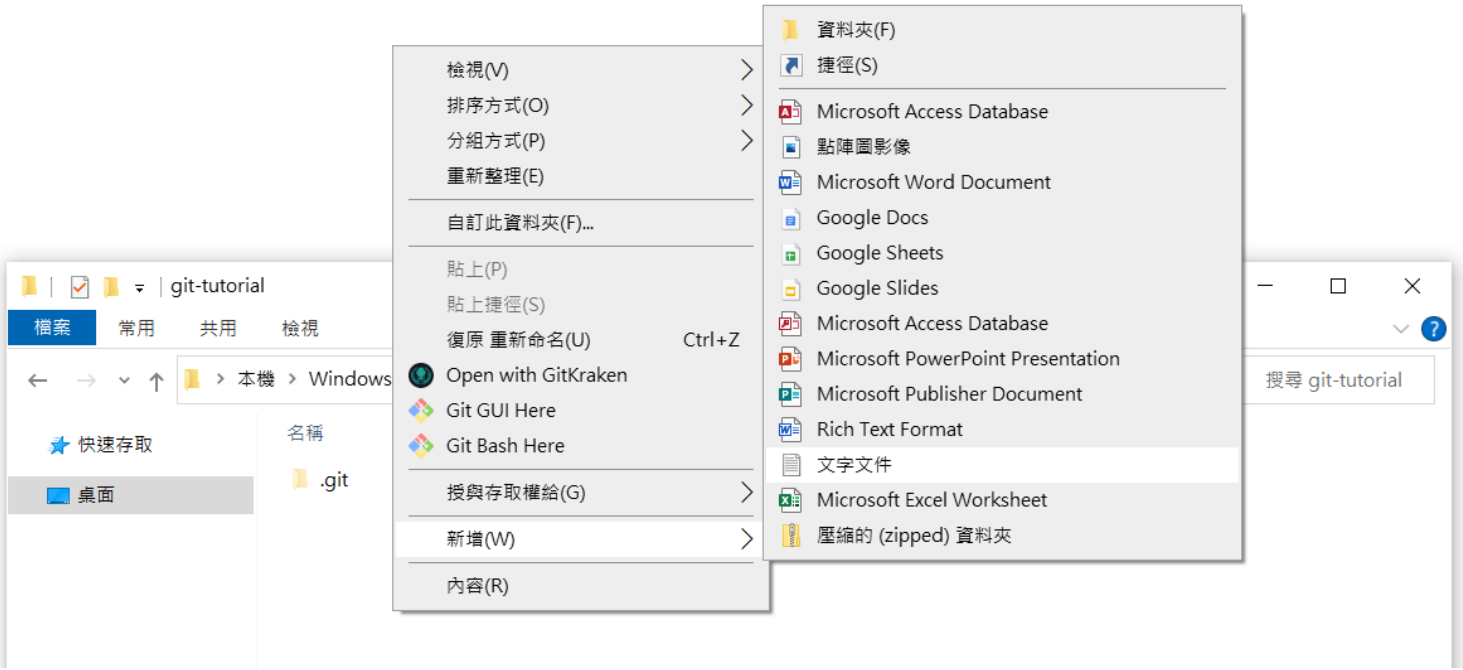
建立與連結遠端儲存庫(Remote Repository)3

- 連結GitHub，將會看到遠端儲存庫也成功地被建立出來。



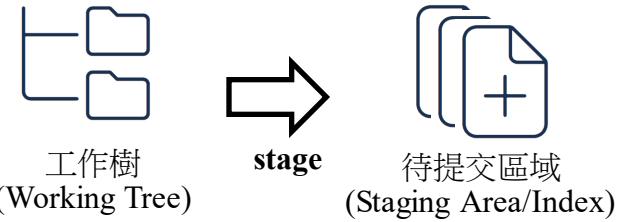
檔案至本地端儲存庫

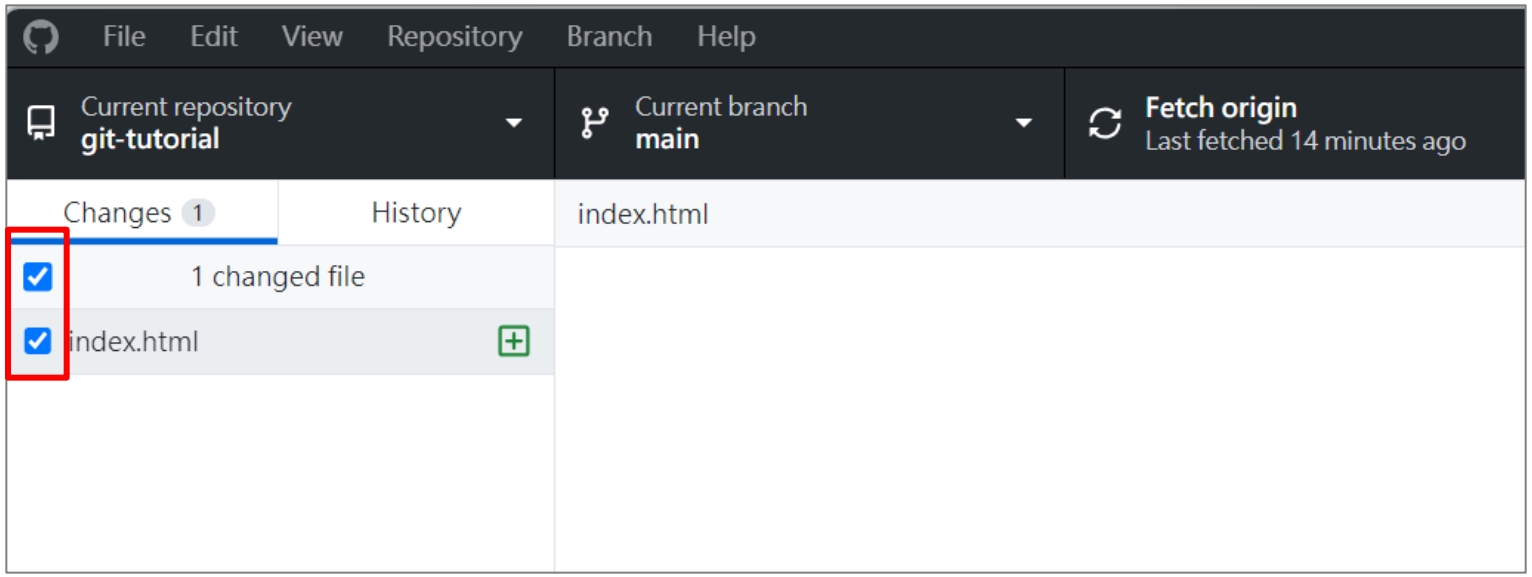
- 加入**未追蹤檔案**至工作目錄
  - e.g., *index.html*



檢視檔案狀態

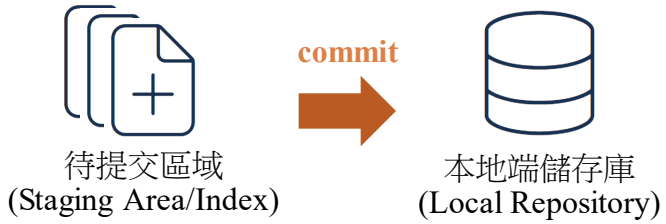
GitHub Desktop會預設勾選所有新加入檔案，以讓我們將未追蹤檔案加入待提交區域

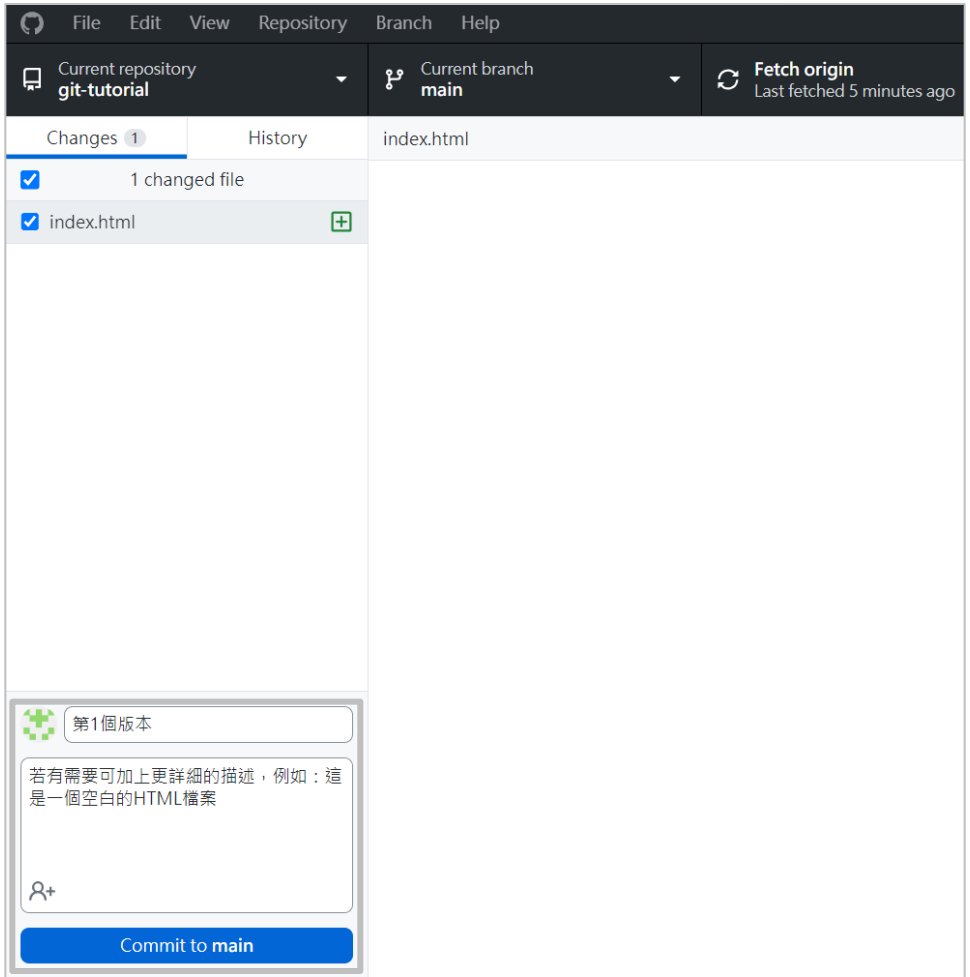




提交檔案至本地端儲存庫1

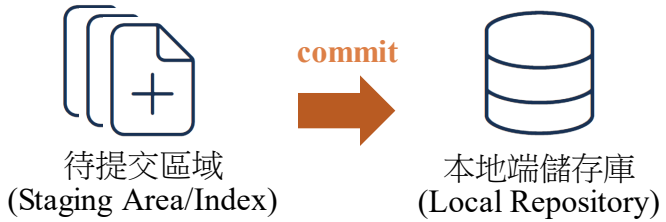
- 我們要撰寫提交訊息(commit message)，實際建立commit。

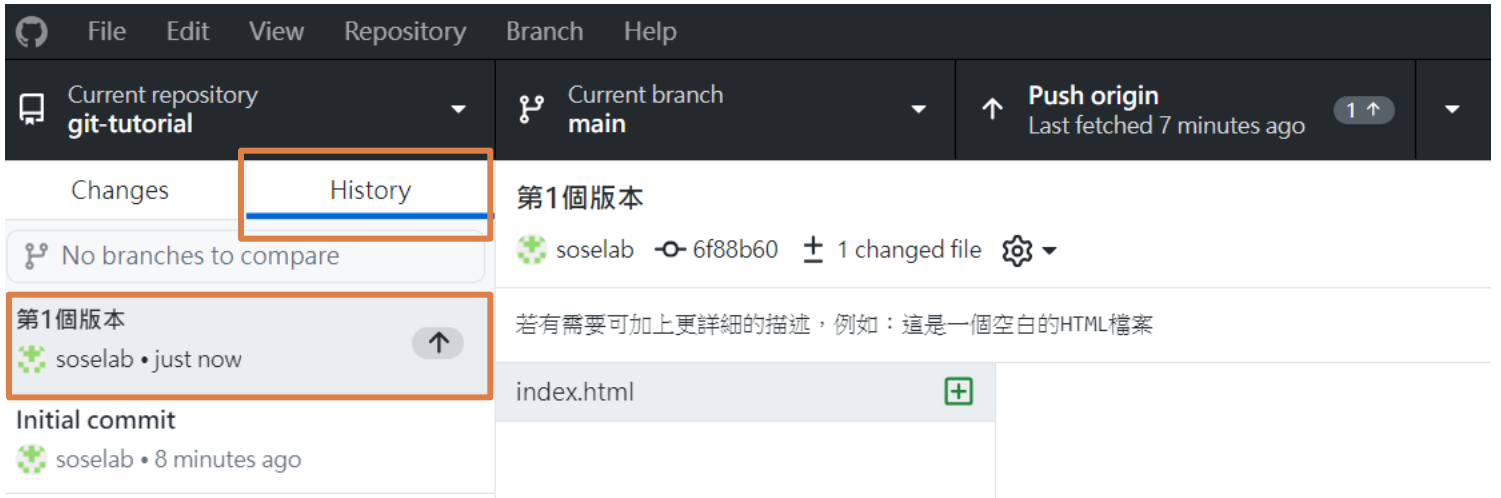




提交檔案至本地端儲存庫2

- 檢視提交歷程(history，也就是log)

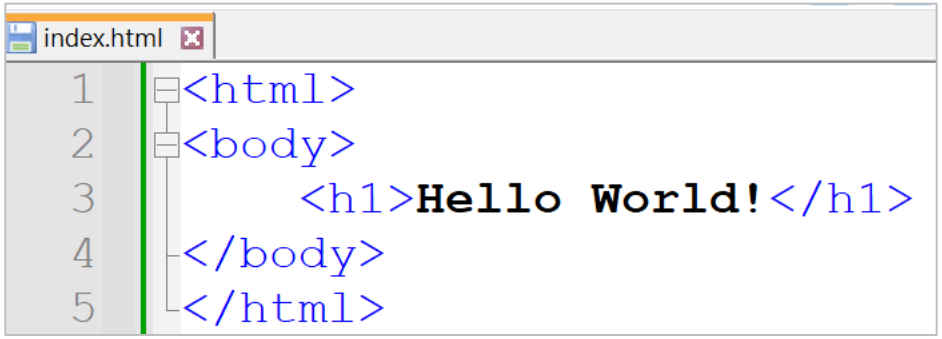




小量改進 - 修改內容

- 修改/變更工作目錄(working tree)中的*index.html*

```
<h1>Hello World!</h1>
```



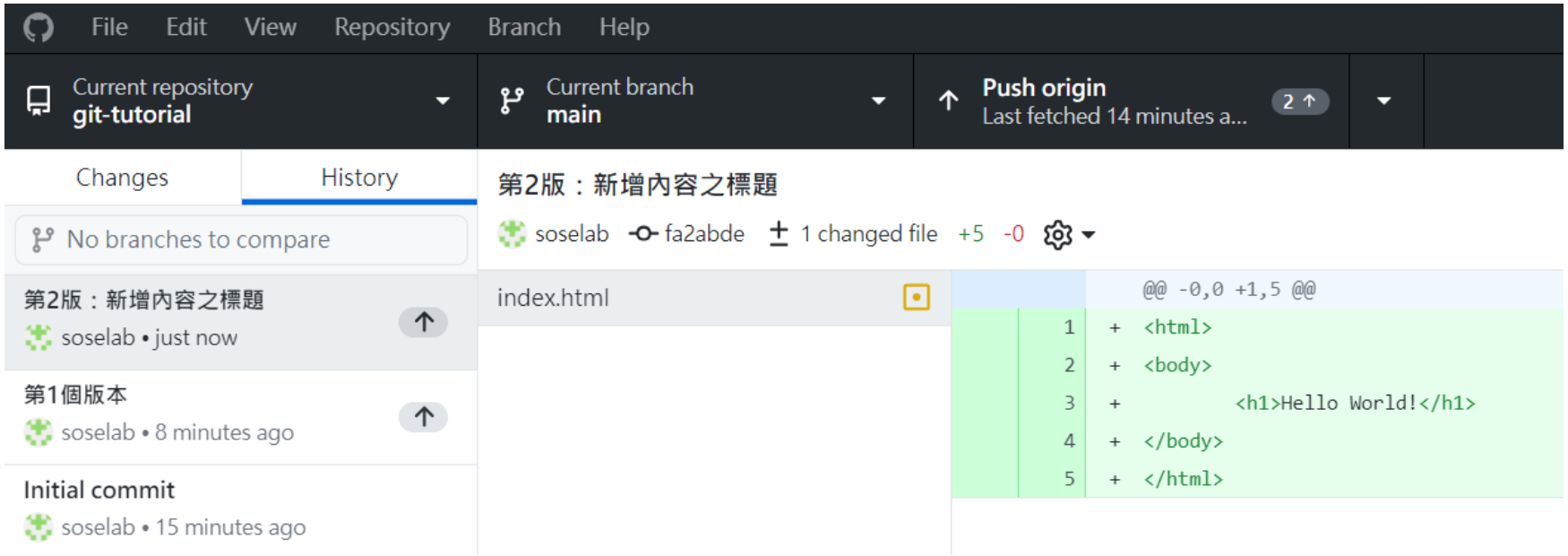
(Notepad++)



(Chrome)

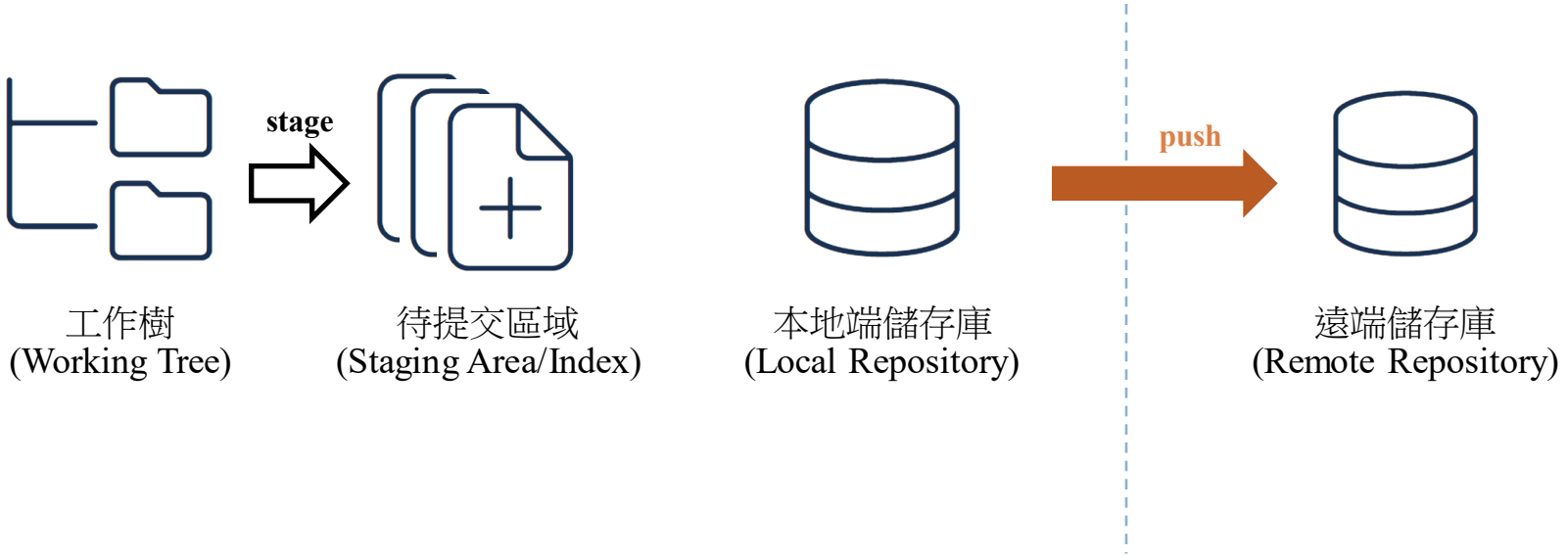
小量改進 - 修改內容

- 檢視提交歷程



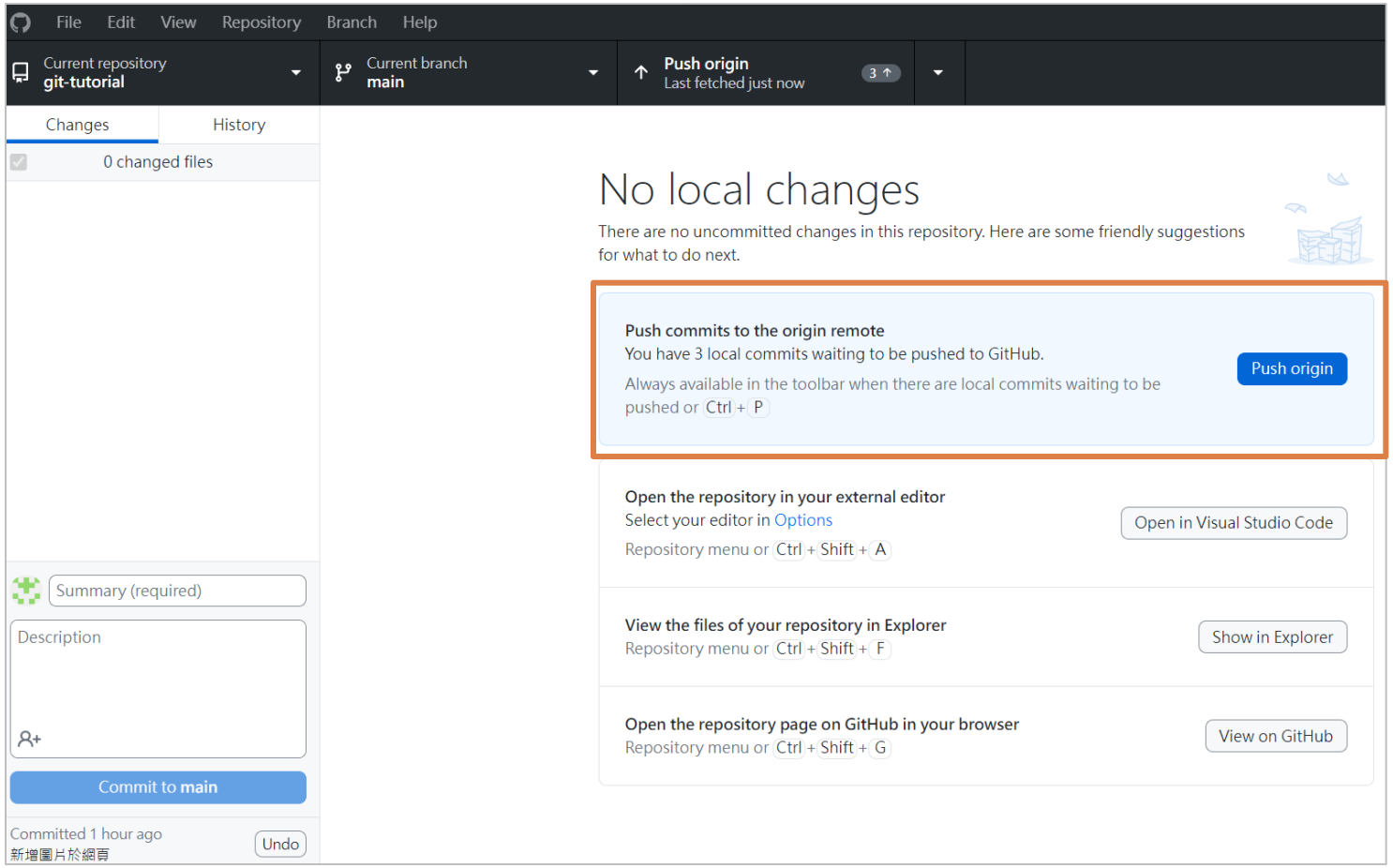
同步遠端儲存庫

- 我們可將本地端儲存庫的一個提交歷程**推送(push)**至遠端儲存庫



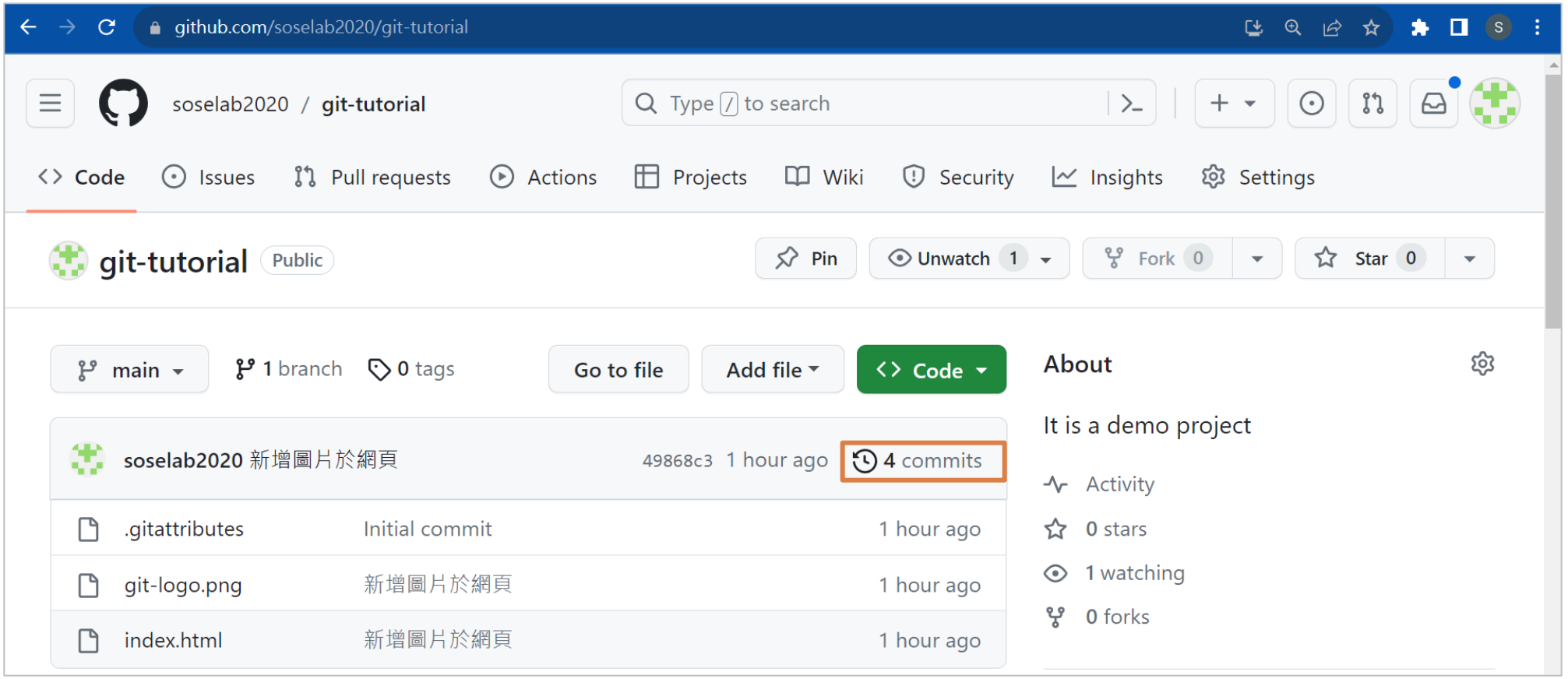
同步遠端儲存庫

- 點選[Push origin]

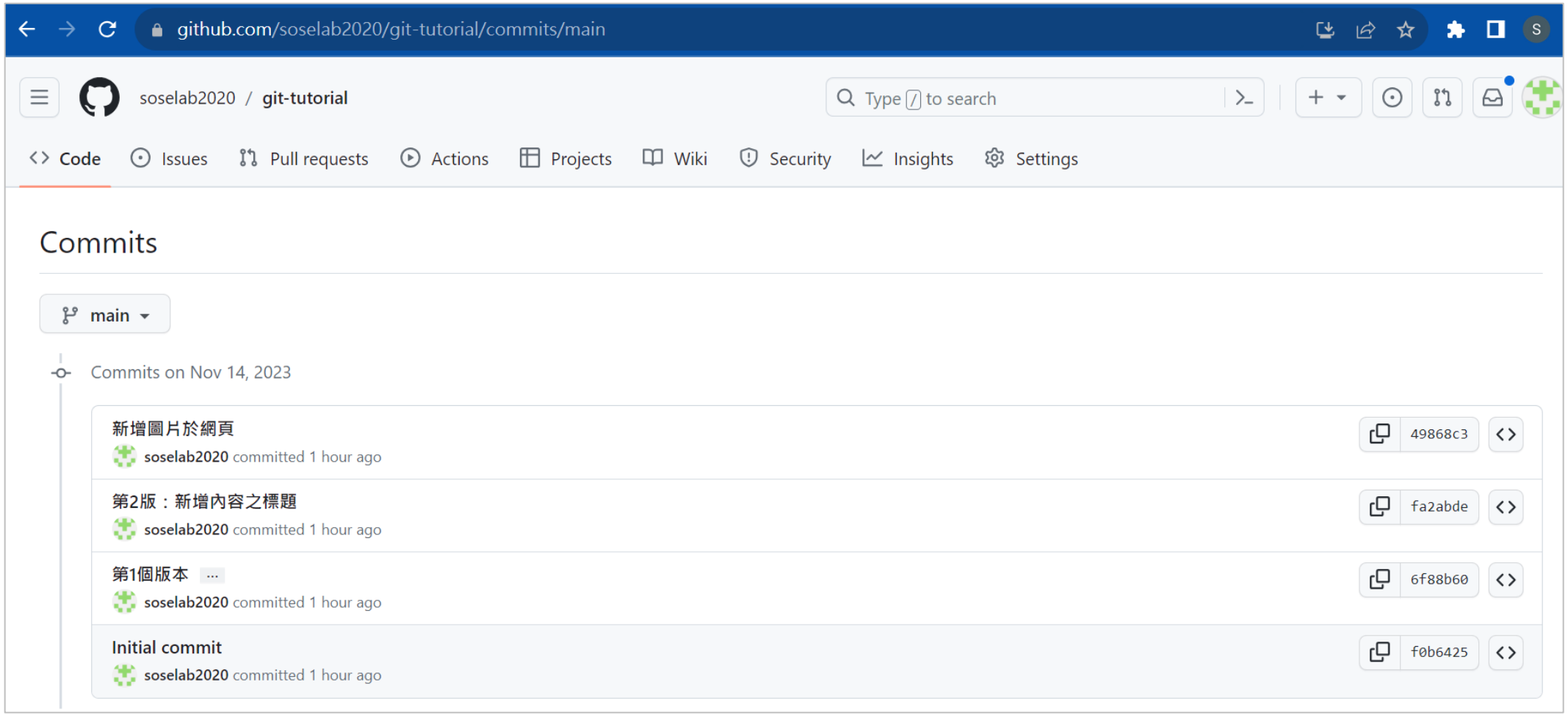


完成遠端儲存庫同步

- 重整遠端儲存庫
  - 可看到所有commit均以推送至遠端儲存庫



查看遠端儲存庫上的提交歷程



Any Question?

